Molarium: Agent-Generated Adaptive Software for Molecular Design

A perspective on user-directed adaptive software, persistent molecular state, and scientific reproducibility in drug discovery

Rafal P. Wiewiora
PsiThera
rafal.wiewiora@psithera.com

Woody Sherman
PsiThera
woody.sherman@psithera.com

September 6, 2026

Abstract

Commercial drug discovery software has traditionally bundled three sources of value: the interface, the scientific models, and the specialized expertise required to run the models through the interface, thereby translating outputs into design decisions. Large language models, coding agents, and tool-using agents are beginning to unbundle this paradigm. Interfaces, analyses, workflow logic, and substantial portions of scientific software can increasingly be generated to meet the needs of a particular scientist, project, and question, while agents can automate sophisticated workflows that once required specialist users.

This shift repositions where competitive advantage is created in computational drug discovery. More intuitive interfaces and faster execution alone do not fix model inaccuracies or broaden the applicability domain. As the underlying tooling becomes increasingly accessible and fluid, strategic differentiation will shift away from familiarity with a particular software package and toward superior preditive algorithms, more generalizable models, proprietary experimental data, accumulated discovery intelligence, and the human discernment needed to frame the right questions and evaluate the answers.

Molarium, available at https://molarium.org, provides a case study of this transition in the context of molecular design software. Created during an intensive weekend effort of a tireless drug-discovery scientist and an AI coding agent, this open-source molecular workbench operates exclusively in a standard web browser. It integrates local molecular graphics, editing, conformer generation, energy minimization, molecular dynamics, machine-learned potentials, and protein-structure prediction workflows. Beneath its rapidly adaptable interface, Molarium maintains a versioned molecular state together with user actions, method specifications, provenance, and validation evidence. Decoupling an adaptable interface from an enduring scientific core points toward a broader architecture for generated adaptive software in drug discovery.

Keywords: agentic software development; computational chemistry; molecular design; WebGPU; scientific validation; provenance; discovery intelligence

1 From fixed software products to generated adaptive software

In computational drug discovery, success depends on deploying the right methodology at the right time, supported by appropriate parameters, explicit assumptions, and sound scientific context. Historically, this work has been concentrated among computational specialists and has required substantial operational overhead. Docking depends on protein preparation, protocol selection, constraints, and pose evaluation. Molecular dynamics requires complex system setup, solvation, ion placement, force-field assignment,

compute infrastructure, equilibration, production simulation, potential for enhanced sampling, trajectory analysis, visualization, and interpretation in the context of project-specific decisions. Free-energy calculations add further complexities to molecular simulations with alchemical perturbations, sampling windows, convergence assessment, error analysis, and alignment with experimental conditions. Comparable layers of infrastructure and expertise surround cheminformatics, quantum chemistry, ADME property prediction, generative molecular design, and protein structure prediction.

Historically, commercial drug discovery software has created considerable value by simplifying and integrating many of these activities. Graphical interfaces reduced technical barriers, workflow engines connected fragmented programs, and software packages encoded operational details needed to apply methods consistently. A single static inferface (perhaps updated quarterly) coupled scientific questions to a fixed software environment. User-specific requests competed with broader corporate product roadmaps that attempted to balance the needs of a diverse user base. Even when feasible, implementing new methods from the literature or integrating open-source software often required software engineering support. Scientific users adapted their questions to the capabilities and conventions of the available tools within their preferred/available graphical interface rather than adapting the software to their needs.

Large language models and coding agents are beginning to change this relationship. A scientist with deep drug-discovery expertise but limited software programming experience can increasingly describe an application, analysis, visualization, or workflow in natural language and have software constructed around the scientific need. Tool-using agents can operate established methods, build complex workflows, implement methods from the literature, handle failures, assemble results into insights that support decisions, and even optimize processes based on iterative feedback. Earlier systems such as ChemCrow and ChemGraph established proof-of-concept for language models coordinating chemistry tools and computational workflows [1–4]. While their practical scope was limited compared with current coding agents, they helped establish the feasibility of coupling language models to executable scientific tools.

The consequence is a shift in how drug hunters can interact with software. Scientists can increasingly customize software around project-specific questions, available data, and their preferred way of reasoning about a problem. A medicinal chemist optimizing a macrocycle may need a different interface and set of calculations than a structural biologist testing a mechanistic hypothesis or a program team deciding which compounds to advance from a virtual screening campaign. In this model, the interface becomes an adaptive and potentially transient view over a more durable scientific substrate. Changes to the interface do not imply changes to the underlying record of the work: molecular states, computational methods, parameters, intermediate results, user actions, and software versions should persist and link through an auditable provenance history. This separation between a mutable graphical layer and a persistent scientific state is important for reproducibility, inspection, and reconstruction of prior decisions. Although Molarium presents as a relatively simple browser-based application, it is built on a substantially richer underlying architecture for managing versioned molecular state, tracking computational provenance, and executing adapted implementations of established molecular modeling algorithms efficiently within the constraints of a local web browser.

Molarium's development directly reflects this paradigm. It began as a need for a better environment to view and modify molecular structures; the first version evolved through active scientific use over a weekend. Scientific problems drove software changes: unrealistic geometries led to refined protocols; unconverged energies prompted explicit numerical reference checks; ambiguities about execution backends led to clearer identification of the active engine; and unvalidated methods became explicit boundaries rather than implicit assumptions. The software evolved with the scientific questions it was built to address. Appendix A describes the execution architecture, numerical validation, and an executable example of this design history. Appendix B describes scientific workflows for human users and agents, pairing interface instructions with agent skill contracts and public API examples, and clarifying the requirements, outputs, and limitations of each workflow.

2 Molarium and the value of a persistent molecular state

Molarium is a local-first molecular viewer, builder, and simulation workbench released under the Apache License 2.0. Most supported calculations execute on the user's local device using WebAssembly or WebGPU, allowing molecular graphics, editing, cheminformatics, conformer exploration, molecular mechanics, molecular dynamics, machine-learned potentials, and selected protein-structure prediction workflows to coexist in a standard browser environment. The browser reduces installation and orchestration overhead and, for supported local calculations, avoids remote job submission, cloud-compute charges, and application licensing while keeping computation inside the interactive design loop. WebAssembly provides a route for mature compiled scientific libraries to run locally, while WebGPU exposes modern accelerators through a portable browser API [5, 6].

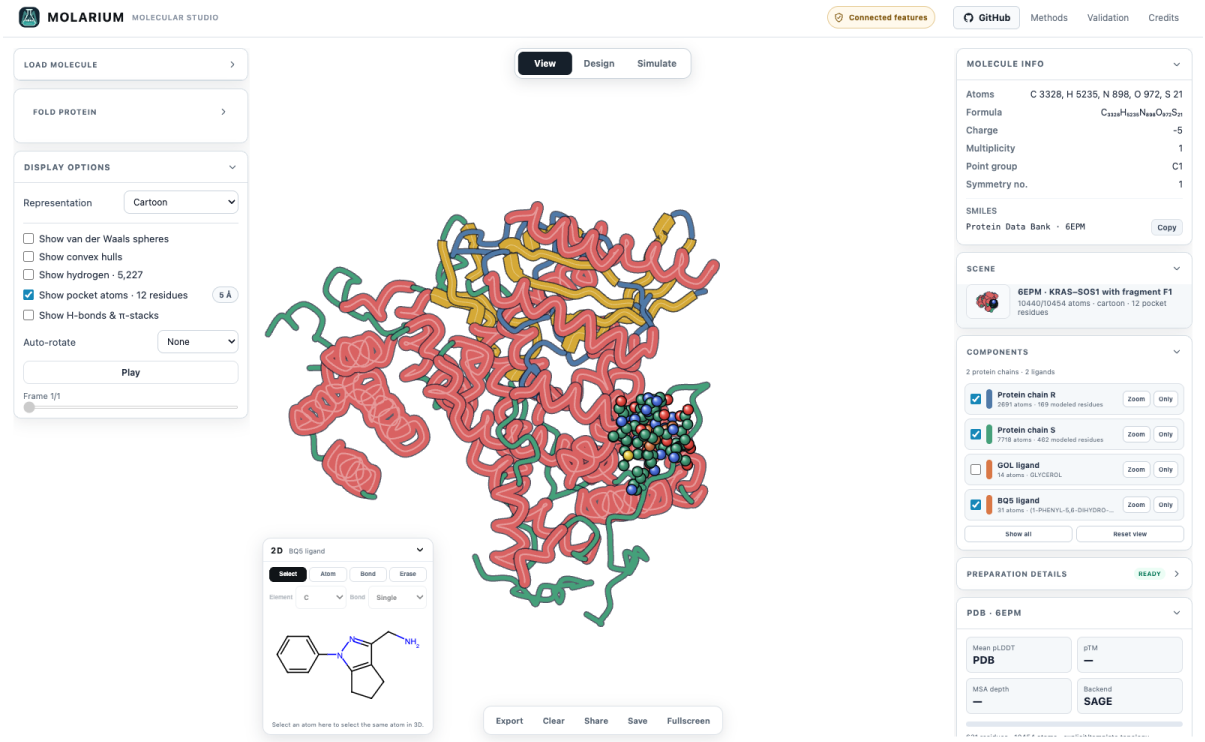

Figure 1: Molarium in its normal View workspace after browser-local Sage/CCD preparation of the experimental fragment F1-bound KRAS–SOS1 complex (PDB 6EPM). The official RCSB Chemical Component Dictionary topology supplies the BQ5/F1 bond orders and the chemically correct 2D depiction shown at lower left. The complete deposited protein assembly is shown as cartoons, while BQ5/F1 and protein residues within 5 Å are rendered atomically to localize the allosteric design pocket. Solvent, cofactors, and hydrogens are hidden for clarity. This scene provides structural context; the reference-informed SOS1 replay in Fig. 2 uses 5OVE/AXE as its sole coordinate-bearing starting input.

One of the advanced architectural capabilities is the preservation of a common scientific state. Molarium maintains explicit topology, coordinates, protonation state, chemical edits, preparation history, trajectories, model identity, and provenance as properties of the molecular session rather than allowing each view or calculation to construct its own interpretation of the molecule. The interface can therefore change rapidly without redefining the scientific object beneath it.

Three task-specific views illustrate this separation. "View" emphasizes the molecular system, including molecular representations of protein, ligand, waters, ions, contacts, measurements, and selected residues. "Design" exposes chemical and geometrical modifications. "Simulate" emphasizes preparation, simulation, and analysis. The controls differ because the scientific questions differ; topology, coordinates, method identity, and history remain attached to the same evolving molecular state. Detailed implementation and

replay examples are provided in Appendix A, while the state mutations, exports, and failure behaviour of the corresponding user-facing modules are specified in Appendix B.

The SOS1 replay expresses a molecular-design procedure as an executable program. Its public API actions specify graph edits, intended contacts, allowed atomic motions, candidate evaluations, and selection rules. The designer defines an AWW ligand direction and Tyr884 contact; a separate calculation tests the Phe890 response while holding that ligand pose fixed. The BAY-293 graph rewrite then retains a declared distal spatial feature. This separates the supplied hypothesis from the calculated response and records the alternatives examined, not only the final structure. A contact, permitted motion, or selection rule can be changed explicitly to define a new test. Appendix A pairs pseudocode for the complete movie with a human-readable account of the design decisions and outcomes; the full executable API JSON remains available. The AWW and BAY-293 ligand RMSDs are 0.851 and 1.880 Å, respectively, after receptor alignment and graph-symmetry minimization without ligand fitting. The crystal series informed the hypothesis; this is not a blinded prediction.

Three complementary views are available at `https://molarium.org/sos1`: the recomputable replay runs the recorded operations through the public API; the precomputed replay allows inspection of seven exact saved checkpoints without calculation; and the MP4 records that checkpoint review with brief calculation-status popups. Recalculation can change numerical results, whereas the saved states preserve the reported result. Neither the movie nor its shortened popups represent calculation speed; Appendix A gives the protocol and evidence boundary.

This separation between interface and molecular state becomes important when interfaces are inexpensive to modify. If every generated view can redefine a molecule, select a different charge model, change protonation, alter preparation, or reinterpret a trajectory, flexibility becomes a source of scientific ambiguity. Generated adaptive software therefore requires a stronger boundary between presentation and scientific state than a conventional fixed interface typically provides.

Molarium extends this principle by preserving the sequence of molecular states and user actions that produced the current state. Edits, selections, manipulations, calculations, and other interactions form an event history rather than leaving only the final coordinates. The resulting architecture is closer to version control than to a traditional molecular project file. Each molecular state can be associated with the actions that produced it, the calculation applied to it, and the evidence available at that point in the design process.

This concept is analogous to "Git for molecules", although a drug design history is richer and more complex than tracking revisions of source code. A design history includes structural changes together with evolving hypotheses, model outputs, uncertainty, and experimental evidence. In a discovery setting, such a record could capture which analogs were considered, which structures or properties influenced a design, which alternatives were rejected, and what was learned when a compound was synthesized and tested. This creates a foundation for recording medicinal chemistry design trajectories rather than storing only the compounds that survived them. Much of the information that explains how a program learned lies in alternatives that were considered and abandoned, yet conventional compound databases usually only preserve experimental endpoints.

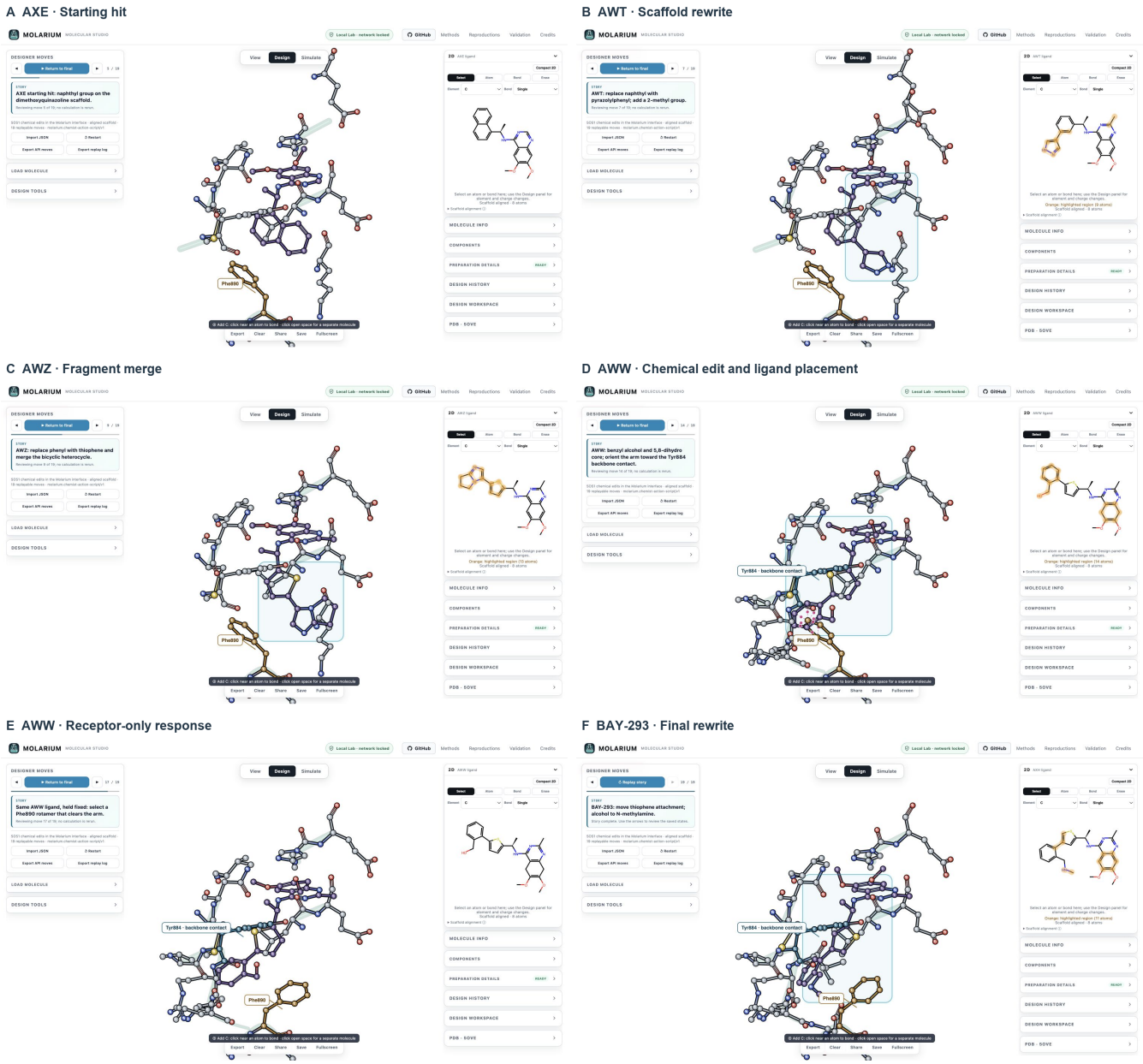

Figure 2: Chemical edits and receptor response in the Molarium interface. Each panel is a full-tool screenshot with the ligand graph in the enlarged 2D panel. (A) AXE starting hit. (B) AWT: replace naphthyl with pyrazolylphenyl and add a 2-methyl group. (C) AWZ: introduce thiophene and merge the bicyclic heterocycle. (D) AWW: install benzyl alcohol and the 5,8-dihydro core, then orient the arm toward the Tyr884 backbone-carbonyl contact. (E) Hold the same AWW ligand fixed while selecting a Phe890 rotamer that clears the arm. (F) BAY-293: move the thiophene attachment and replace the alcohol with an N-methylamine. Orange 2D highlights mark added atoms and local chemical changes relative to the preceding structure; equivalent Kekulé representations and reassigned atom identifiers do not count as changes. The AWW highlights persist through ligand placement and are cleared for the receptor-only panel. All panels use exact saved checkpoints in the same pocket view. The crystal series informed the design hypothesis, but later crystal coordinates were not used to generate these states.

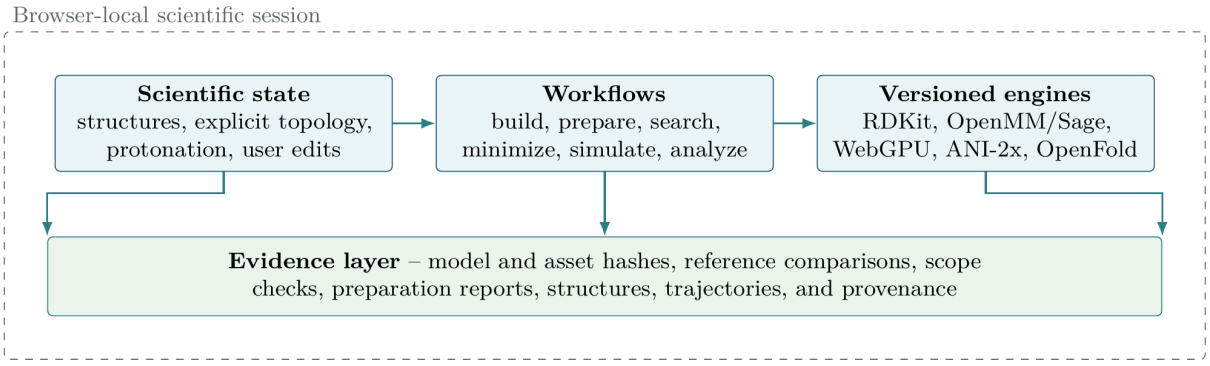

Figure 3: Molarium's local-first architecture. Named computational engines act on an explicit molecular state while provenance and validation evidence remain attached. The same principle provides a foundation for versioned molecular histories in which scientific actions can be replayed and compared without allowing a new interface to redefine the underlying state.

The browser also provides a useful trust boundary for proprietary work. Molarium's "Local Lab" mode can restrict network access and verify reviewed application assets while keeping supported calculations on the local device. Because privacy controls depend on deployment configuration, this architecture makes data movement explicit and auditable. Organizations can evaluate and enforce how molecular data are handled rather than treating transfer to remote infrastructure as an invisible property of the software stack. Appendix B specifies the connected and Local Lab network policies and makes clear that local calculation and fully offline operation are distinct claims.

3 Scientific contracts: provenance, validation, and falsifiability

The emergence of rapidly adaptive software introduces a scientific vulnerability. Agentically generated workflows become especially risky when ease-of-use obscures the underlying scientific methods. As the time required to build a custom analysis view drops, that view still needs to expose the molecule, model, parameters, assumptions, and evidence that produced the result. Reducing the friction of software development without those safeguards increases the rate at which scientific ambiguity and errors can be introduced.

We use the term "scientific contract" for the collection of invariants and evidence that gives a computational output its meaning. A molecular mechanics result cannot be defined by the generic label "molecular mechanics." Its identity depends on the molecular state, charge assignment, parameterization, energy function, treatment of nonbonded interactions, boundary conditions, constraints, integrator, numerical implementation, and precision. A learned potential depends on its architecture, weights, preprocessing, supported chemistry, and training domain. A protein-structure prediction depends on checkpoint selection, sequence and template information, inference protocol, and other model-specific choices. The surrounding interface may adapt dynamically, but these scientific identities must remain explicit and protected from silent drift.

Molarium preserves distinctions among computational pathways even when they operate in the same workspace. RDKit supports chemical perception and conformer generation; OpenFF Sage defines a molecular-mechanics model; OpenMM Reference provides an independent numerical route for selected validation tests; STORMM-inspired WebGPU code advances homogeneous replicas; ANI-2x provides a machine-learned potential with a defined chemical domain; and OpenFold supports a separate protein-structure prediction pathway [7–12]. These methods can be compared or chained without implying that their energies, assumptions, or domains are interchangeable. Appendix A specifies the implemented numerical pathways and their validation boundaries; Appendix B records how those pathways are exposed, invoked, and constrained in the current workbench.

The same principle applies to workflow provenance. Reproducible computational science requires retention of the data, computational process, execution environment, and provenance that produced a result [15, 17]. More recent work formalizes workflow invariants and testable reproducibility criteria rather than treating provenance as passive metadata [16]. Generated adaptive software increases the importance of this requirement because the workflow itself may be created or modified during the scientific interaction rather than existing as a single release version defined in advance.

This leads to a stronger requirement: generation of a scientific workflow should be accompanied by generation of the conditions under which its claims could be falsified. If an agent implements a conformer-search strategy, the output should include the benchmark that tests whether conformational coverage improves, the independent reference used for comparison, the quantitative tolerance that defines agreement, the applicability limits of the test, and explicit failure conditions. If an agent adds a numerical kernel, it should generate the corresponding independent comparison and acceptance criteria. These are better understood as scientific contract tests than as ordinary software unit tests because they define what scientific claim the implementation can support.

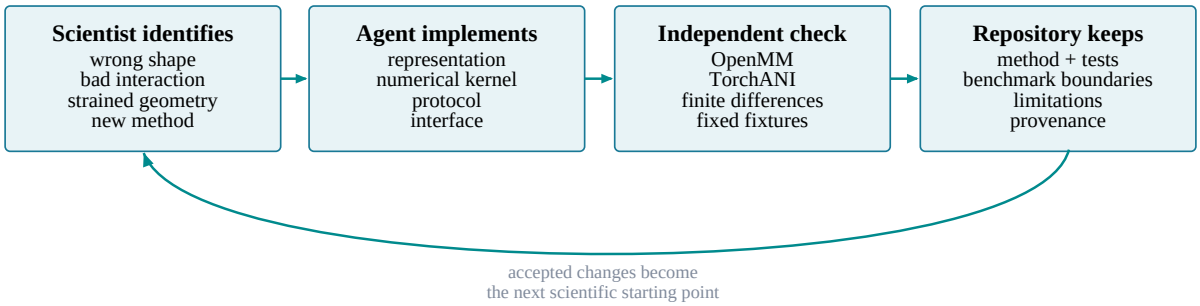

Figure 4: The scientist–agent build loop. Scientific needs and objections are translated into software changes, while each new capability is paired with claim-defining tests, reference evidence, applicability limits, and provenance. The ability to generate a workflow and the ability to specify how its claim could fail are treated as parts of the same scientific result.

Molarium's development repeatedly followed this pattern. A chemically implausible geometry exposed a problem in representation and editing semantics, and a questionable energy prompted comparison with an independent implementation. Similarly, a discrepancy between CPU and GPU execution revealed both a unit error and an incorrect assumption about which backend was running. Requesting constrained dynamics led to implementation of SHAKE/RATTLE and comparison against a reference pathway [13, 14]. In each case, the useful scientific result consisted of the feature together with a test that established the boundary of the claim.

Fluent agentic software makes this distinction more consequential, since problems are not always obvious and do not necessarily result in errors. For example, a unit error can still produce a smooth trajectory and independent implementations can agree if they inherit the same flawed inputs. Successful execution and an attractive visualization therefore occupy the bottom of an evidence hierarchy rather than establishing that a molecular prediction is correct.

When software construction is difficult, the labor of building a computational method can obscure the separate obligation to validate it. As implementation becomes easier, the scientific responsibility becomes harder to ignore. The workflow, its provenance, and the conditions that could falsify its claims should therefore be treated as a single scientific object.

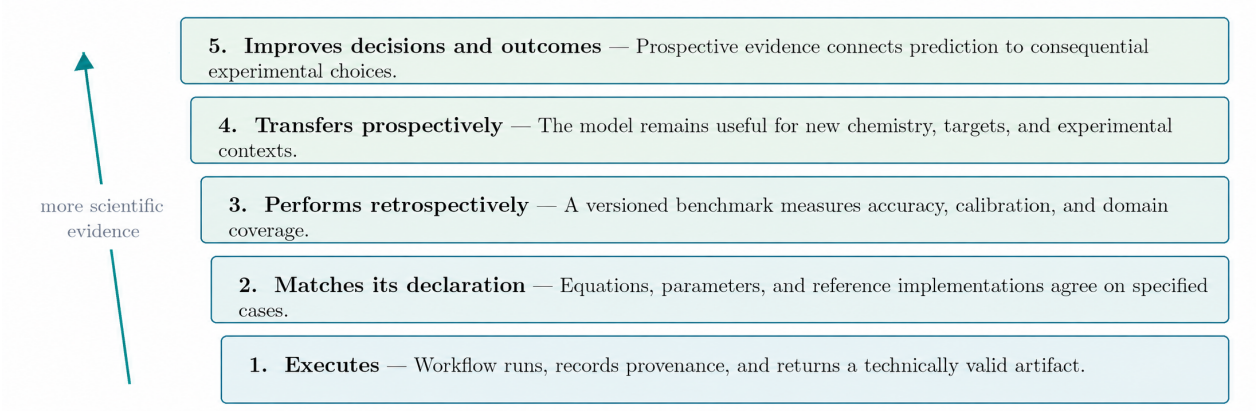

Figure 5: An evidence ladder for generated scientific software. Successful execution establishes less than numerical agreement; numerical agreement establishes less than retrospective predictive performance; retrospective performance establishes less than prospective transfer; and prospective accuracy is distinct from measurable improvement in drug-discovery decisions.

4 Local computation changes the interaction model

A web browser cannot always replace production molecular simulation software or high-performance computing. Large periodic systems, long trajectories, and large-scale screening studies can require substantially more compute resources than a personal machine can provide. Molarium currently addresses calculations that are sufficiently bounded to remain inside the scientist's interactive design session, although remote job submission to a HPC cluster or the cloud for specific calculations would be trival to add.

The practical benefit of local compute is the reduced decision latency. Consider a medicinal chemist examining a bound ligand and asking whether a proposed three-dimensional edit creates an avoidable steric problem or opens a plausible alternative conformation of the protein binding pocket. In a fragmented workflow, the scientist may export coordinates, request or run a separate calculation, move results back into a viewer, and reconstruct the context in which the question arose. In Molarium, the editing, relaxation, conformational exploration, and trajectory inspection all run locally within the same session, tracking the molecular state, the calculation, associated parameters, the design question, and decisions.

Local execution also changes economics for these interactive calculations. The marginal cost of many small calculations approaches the cost of hardware already on the user's desk, without requiring a remote orchestration cycle or a separate software license for each supported operation. This advantage is most relevant in the rapid design loop, where a calculation may be useful because it answers a focused question in minutes rather than because it achieves the maximum possible throughput on a benchmark.

The lower operational barrier increases the importance of defining the project questions appropriately and choosing calculations deliberately. A medicinal chemist deciding between two substituent orientations may gain value from a constrained local refinement that changes which analog is synthesized next, while running hundreds of additional simulations after the decision is already insensitive to the result adds computational activity without increasing scientific leverage. The operational efficiency helps the scientist focus on the right question, apply the appropriate method, and prioritize molecules that have a realistic chance to advance the program.

Local execution also redistributes control. The scientist can determine which analysis should exist, how data should be juxtaposed, and which calculation should be available at the point of decision. Molarium illustrates this shift: a scientist did not need to wait for every useful workflow to appear on a conventional product roadmap. Missing interfaces, algorithms, and methods can be added while the underlying question was still active.

5 What becomes scarce when the tool layer becomes cheap

A concrete medicinal-chemistry example helps separate the layers of value. Suppose a project team is considering a substitution intended to improve a ligand's interaction with a newly identified pocket while preserving permeability. The tool layer provides the editor, visualization, coordinate handling, workflow logic, and connection to the relevant calculations. The model layer determines whether the predicted pose, conformational preference, interaction energy, or permeability estimate is credible. The discovery-intelligence layer supplies the project-specific context: prior SAR, assay conditions, structural interpretation, known liabilities, synthetic constraints, and the reason the team is considering that particular design at that point in the program. Coding agents can make the first layer easier to construct. The value of the second and third layers depends on scientific accuracy and accumulated knowledge rather than interface complexity.

The tool layer includes graphical interfaces, visualization, format conversion, wrappers, workflow logic, reports, job control, data plumbing, and bounded numerical implementation. These capabilities remain important and require maintenance, but their marginal reproduction cost is falling quickly as coding agents improve. A scientist can increasingly generate a project-specific view or connect an existing computational method without waiting for the corresponding feature to appear in a commercial release.

The model layer includes force fields, quantum-mechanical methods, docking and scoring functions, free-energy models, machine-learned potentials, protein-structure predictors, molecular representations, property models, parameters, weights, uncertainty calibration, and the evidence that establishes their useful domain. Easier invocation does not improve predictive quality. A docking model with an inadequate scoring function remains inadequate when an agent invokes it. A systematic force-field error can become more damaging when automated workflows propagate it across thousands of calculations. A machine-learned property model does not gain extrapolative validity because a language model describes its output fluently, and a protein-structure predictor does not become reliable for a particular conformational state because it is available through a better interface.

Differences among models therefore become more consequential as the mechanics of invoking them become less scarce. Drug discovery frequently requires extrapolation into new chemistry, structural states, and biological contexts rather than interpolation within a well-sampled training distribution [18–20]. Faster access can accelerate good models and weak models alike.

The discovery-intelligence layer includes proprietary experimental data, negative results, project history, assay context, structural interpretation, design rationale, internal protocols, expert judgment, and program objectives. Molarium intentionally contains little of this layer because it is a public workbench built primarily from public scientific components. The same architecture could be connected to internal structures, assay data, pharmacokinetics, target biology, medicinal-chemistry reasoning, and program objectives. That project-specific context can determine which computation is relevant and how much weight its output should receive.

Versioned molecular history provides a bridge into discovery intelligence. Traditional compound databases record what was made and measured well, but they rarely preserve what was considered, why one design was preferred, which model or structure influenced the decision, or what evidence was visible at the time. A persistent action history can capture that decision trajectory.

For example, a chemist might propose adding a substituent after inspecting a pocket in a protein–ligand structure. The project record could retain the parent structure, the molecular edit, the structural hypothesis, the conformational or energetic calculations viewed before synthesis, competing designs that were rejected, and the subsequent experimental measurements. If the design later fails because an assumed interaction was absent or a liability dominated the profile, that information remains associated with the decision that created the molecule. Future scientists and agents can then ask which assumptions repeatedly led to productive designs and which repeatedly failed. This is decision provenance: preserving the

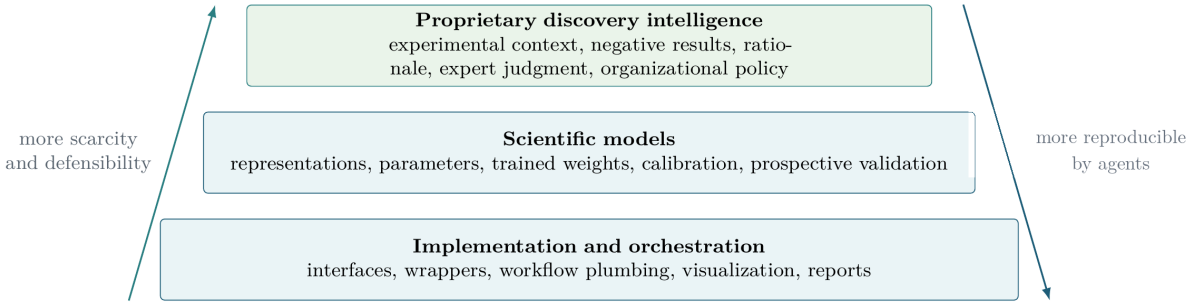

Figure 6: Three interdependent layers of value in generated adaptive molecular design. Implementation and orchestration are becoming increasingly reproducible by coding agents. Scientific models remain differentiated by accuracy, representation, domain coverage, calibration, and validation. Proprietary discovery intelligence combines experimental data with the context and decision history that make those data useful.

relationship among hypotheses, interventions, predictions, decisions, and outcomes rather than storing only final compounds and assay values.

Agentic systems provide a mechanism to operationalize some of this accumulated knowledge through scientific skills. Here, a skill means a version-controlled, machine-executable procedure that combines a defined tool or model with an organization's accepted inputs, assumptions, validation criteria, applicability limits, failure handling, and escalation rules. A protein-preparation skill might encode how crystallographic waters, protonation states, alternate conformations, unusual residues, and ambiguous cases are handled. A free-energy skill might define accepted perturbations, convergence criteria, diagnostics, and failure modes. The durable asset is the scientific procedure and its validation, rather than the natural-language prompt used to invoke it.

6 The changing role of scientific expertise

As software becomes easier to construct and operate, scientific judgment becomes more valuable. Traditional computational expertise includes substantial operational knowledge: preparing inputs, locating settings, translating files between programs, recovering failed jobs, and reproducing standard analyses. An increasing fraction of this work can be encoded in software and agents. The broader automation literature has long recognized that reducing routine operation can increase the importance of human expertise when systems encounter ambiguous or abnormal conditions [21].

The highest-value expertise shifts toward selecting the right question, choosing an appropriate level of model fidelity, defining acceptable tradeoffs between cost and accuracy, recognizing implausible results, selecting meaningful controls, and determining which evidence would change a project decision. Scientists closest to the underlying discovery problem—whether medicinal chemists, structural biologists, mechanistic biologists, pharmacologists, or computational scientists—can increasingly interact directly with relevant data and calculations rather than routing every question through a software intermediary.

This shift changes the leverage of computational specialists. Less specialist time needs to be spent constructing repetitive interfaces, transferring files, or manually operating routine workflows. More can be spent on model development, molecular physics, parameterization, sampling, uncertainty, validation, and difficult edge cases where established workflows fail. Once that expertise is made explicit, part of it can be encoded into validated scientific skills that broaden access without implying that the underlying science has become simple.

Molarium's development illustrates this division of labor. The coding agent could inspect a repository, refactor software, implement numerical kernels, add tests, and repeat the loop rapidly. The scientist

had to recognize chemically implausible results, decide whether a reference was genuinely independent, determine whether a discrepancy mattered, choose acceptable approximations, and identify what evidence would change a decision. The agent compressed the distance between recognizing a scientific need and changing the software; scientific judgment determined which changes mattered.

The bottleneck therefore moves toward epistemology: deciding which approximation is acceptable, whether a faster method sacrifices information that matters to the decision, when an independent control is required, whether a reference is truly independent, whether a model is operating outside its validated domain, what uncertainty must remain visible, and which experiment would discriminate between competing hypotheses. Faster implementation increases the value of asking the right question.

7 Limitations and outlook

Molarium is a case study rather than a controlled experiment in software productivity or drug-discovery performance. Its development involved a small number of scientists, one evolving codebase, and a particular coding-agent workflow. The project did not include a randomized comparison with a conventional software-development team, and it cannot establish a universal productivity multiplier or determine what fraction of commercial scientific software can be regenerated reliably.

The open-source repository can nevertheless support a useful community experiment. Contributors can identify computational capabilities described in papers or public specifications, attempt independent agent-assisted reconstructions, test them against appropriate references, and record which parts can and cannot be reproduced reliably. Over time, this could become a living benchmark for the regeneration of molecular-modeling software, with success defined by scientific contract tests rather than by code generation alone.

Molarium's scientific scope is deliberately bounded. Current browser-native molecular mechanics does not reproduce the full capability of mature native simulation environments. General periodic simulation, particle-mesh Ewald electrostatics, barostats, membranes, unrestricted protein parameterization, arbitrary plugins, and other advanced capabilities remain outside the current browser-native scope. ANI-2x remains valid only within its declared chemical domain, and browser-based OpenFold support is narrower than general production protein-structure prediction workflows. Generated wrappers and easier access must preserve, expose, and enforce these applicability limits rather than implying broader scientific validity. Appendix A catalogues the demonstrated implementation and numerical boundaries, and Appendix B provides the corresponding operational status, failure modes, and end-user limits.

A second limitation concerns the distinction between implementation fidelity and predictive validity. If a browser implementation reproduces an OpenMM reference energy and force to a specified tolerance, that result establishes that the implementation is executing the declared calculation correctly for the tested case. It does not establish that the shared force field predicts the experimental behavior of a new molecule. Likewise, a conformer workflow can reproduce its intended algorithm and explore more structures without improving recovery of biologically relevant conformations. Implementation validation and model validation answer different questions and require different evidence.

The future suggested by Molarium is one in which the interface between scientist and computation becomes more fluid while the scientific state, models, assumptions, evidence, and decision history become more explicit. Project teams can construct temporary views around specific questions; internal models can coexist with open-source methods and commercial services; and the user experience no longer needs to be owned by the provider of each underlying calculation.

The durable infrastructure in such an environment is concrete. A user should be able to inspect the active molecular state and its history, the exact model and parameter versions used for each calculation, declared applicability limits, validation status, assumptions, provenance, and links between predictions and subsequent decisions. These quantities should remain available regardless of which generated view is open. An adaptive interface can change rapidly because the scientific contract underneath it remains

stable and inspectable.

8 Conclusion

Molarium began as a request for a better environment in which to inspect and manipulate molecules. It developed rapidly into a local-first molecular-design workbench incorporating topology-aware editing, protein preparation, conformer exploration, molecular mechanics, molecular dynamics, learned potentials, protein-structure prediction, browser-native GPU computation, provenance, and reference validation. The speed of that development is technically interesting, but the broader lesson concerns the architecture of scientific software and the location of durable value.

Software can increasingly be generated around the scientific question and the user's specific preferences. The quality of the underlying models, validation, and decision history becomes more consequential as the interface becomes cheaper and more fluid. The scarce assets shift toward trustworthy models, proprietary discovery intelligence, and the human judgment required to know what to ask and what to believe.

Molarium also suggests that molecular-design environments should preserve more than the latest structure. A versioned record of molecular states, user actions, calculations, assumptions, and results can capture the trajectory by which scientific decisions were made. Coupled to explicit scientific contracts and falsification tests, that history can make generated adaptive software more reproducible, extensible, and scientifically accountable.

When software is difficult to construct, implementation effort can hide the distinction between building a method and validating it. When implementation becomes easy, the remaining scientific obligations become harder to ignore. The opportunity is to place adaptable computational capability directly in the hands of scientists who understand the problem while preserving the models, evidence, and decision history that determine whether the answer should be trusted.

The tool layer is becoming cheap and adaptive. Scientific advantage moves toward better models, better discovery intelligence, and better decisions.

AI assistance and availability

GPT-5.6 Sol was used extensively as a coding agent during the development of Molarium under human scientific direction, inspection, testing, and revision. Language-model assistance was also used in restructuring and drafting this manuscript. Appendices A and B were drafted primarily by GPT-5.6 Sol and subsequently revised with AI assistance under human direction, with limited direct human editing. The authors remain responsible for the scientific claims, validation, source attribution, and final text.

Molarium is available at `https://molarium.org`. Its source code and technical documentation are publicly available under the Apache License 2.0 at `https://github.com/rafwiewiora/molarium`. Bundled third-party software, model parameters, force fields, and scientific data retain their separately identified terms. Quantitative benchmark results are scoped by the relevant implementation state, fixtures, model and asset hashes, browser and hardware environment, numerical precision, and timing protocol. Appendix A provides the architecture, numerical validation, and executable design-history evidence underlying these results; Appendix B records the inspected operating surface, module boundaries, failure behaviour, and export/persistence contracts.

Acknowledgments

We thank Pat Walters and Haotian Li for insightful discussions and their ongoing creative support.

APPENDIX A
Molarium architecture, numerical validation, and executable design history

Authorship note for Appendices A and B. These appendices were drafted primarily by GPT-5.6 Sol and subsequently revised with AI assistance under human direction, with limited direct human editing. The authors remain responsible for their content.

This appendix specifies the implementation and validation substrate underlying the architectural claims in the main text. It defines the molecular-state contract shared across chemistry perception, parameterization, numerical execution, visualization, and provenance; describes the OpenMM/WebAssembly and WebGPU execution paths; reports fixed-input numerical validation and workload-dependent performance measurements; and documents the content-addressed replay and decision-history mechanisms used by Molarium. The final section provides an executable SOS1 design-history example. Numerical agreement is treated as implementation validation of a declared calculation and is kept distinct from validation of the underlying physical or statistical model.

A.1 Execution architecture

Molarium maintains a single molecular state keyed by stable atom identifiers across ingestion, chemical perception, parameterization, calculation, visualization, and replay. RDKit provides graph interpretation, conformer generation, SMARTS matching, and ligand-level preparation. A JavaScript SMIRNOFF parameterizer applies OpenFF Sage 2.1.0 to generate an explicit numerical recipe containing particle properties, bonded terms, nonbonded terms, exceptions, and constraints. Preparation fails on missing required parameters rather than substituting inferred terms. The accepted recipe is consumed by the OpenMM/WebAssembly reference path and the direct or replica-parallel WebGPU paths, allowing backend comparisons on identical topology, coordinates, parameters, and atom ordering.

The primary data path is:

PDB or SMILES input
↓
molecular graph, stable atom identities, and coordinates
↓
RDKit interpretation → OpenFF Sage numerical recipe
↓
OpenMM/WebAssembly reference | direct WebGPU | replica-parallel WebGPU
↓
viewer, action audit, calculation record, and design history

The same molecular object remains authoritative as calculations return coordinates, forces, energies, or trajectories. Table 1 summarizes the current implementation.

Internal units are nm, ps, daltons, kJ/mol, and kJ/mol/nm; the interface presents angstroms and kcal/mol where appropriate. Numerical engines execute in reusable Web Workers so rendering and user interaction remain responsive. Requests carry job identifiers, and failed workers are discarded and recreated. Results return as compact coordinate, force, energy, or trajectory arrays mapped to the originating atom identities.

For ordinary small molecules, the browser parameterizer applies Sage 2.1.0 patterns and labelled Gasteiger charges [8, 26]. Prepared Rosemary validation fixtures retain their exported numerical recipes and NAGL charges. CPU/GPU comparisons therefore operate on the same prepared graph and numerical inputs rather than independently parameterized systems.

A.1.1 OpenMM WebAssembly reference

OpenMM 8.2.0 is cross-compiled with Emscripten 6.0.6-git and CMake 3.31.0 [9, 27]. A thin C interface constructs an OpenMM `System` from the serialized numerical recipe and returns energies, forces, minimized coordinates, or

Table 1: Current browser implementation.

Task	Software used	Implementation role
Molecular interpretation	RDKit 2025.03.4, WebAssembly	SMILES/graph handling, ETKDGv3, SMARTS matching, MMFF94, explicit UFF fallback, and Gasteiger charges
Force-field assignment	OpenFF Sage 2.1.0	Generates particles, bonds, angles, torsions, nonbonded terms, exceptions, and constraints
Numerical reference	OpenMM 8.2.0 Reference, WebAssembly	Double-precision energies and forces, L-BFGS minimization, and short dynamics used for fixed-input reference comparisons
Direct local mechanics	WebGPU/WGSL, 32-bit floating point	Single-system force/energy evaluation, OBC2/ACE, minimization, and short dynamics
Replica-parallel mechanics	STORMM-inspired WebGPU engine	Advances independent replicas sharing one topology and parameter layout; paired-word fixed-point accumulation; current limit 512 atoms per replica
State and provenance	JavaScript modules, background workers, Canvas, Web Crypto	Preserves atom identity and records action, calculation, replay, and SHA-256-linked provenance

short trajectories. Source, patches, interface code, and built assets carry recorded fingerprints so the deployed reference path can be associated with a specific implementation state.

The reference configuration is nonperiodic and uses the double-precision OpenMM Reference platform. Supported operations include energy and force evaluation, L-BFGS minimization, short dynamics, optional X–H constraints, 2 fs Langevin-middle integration, a cutoff configuration used for validation, and OBC2/ACE with mbondi2 radii [28].

The WebAssembly build exposed a threading-specific setup failure for constrained dynamics: native OpenMM creates helper CPU threads during constraint initialization, whereas the browser build is single-threaded. A serial patch removes the wait path for this build without modifying native OpenMM.

Fixed-input parity was evaluated in two stages using five poses derived from the 7KPA/D84 complex: two pyrazole poses, two tetrahydropyran poses, and one spiro-ketone negative control. These represent three analogue chemistries from one protein target, not five independent systems. The same C interface linked to native OpenMM Reference and compiled for WebAssembly agreed in vacuum, without constraints or a nonbonded cutoff, to a maximum energy difference of 2.84×10^{-14} kcal/mol and relative force RMS of 3.76×10^{-15}. The same five poses were evaluated using browser Sage WebGPU and OpenMM WebAssembly with matched numerical inputs. Maximum energy differences were 1.24×10^{-4} kcal/mol in vacuum and 2.76×10^{-4} kcal/mol with OBC2; maximum relative force RMS error was 1.16×10^{-5} in both modes. These tests establish implementation parity for the specified fixtures and protocols.

A.1.2 Direct WebGPU mechanics

The direct WebGPU path evaluates the prepared force-field equations in WGSL using 32-bit floating-point arithmetic. Force ownership is atomwise: one invocation owns the output force for one atom, and compact incidence lists identify the bonded terms that contribute to that atom. Pair and valence terms are recomputed where required so each invocation writes its force once, avoiding floating-point atomic updates, which WGSL does not provide [22].

The deployed nonbonded representation stores excluded and specially scaled pairs explicitly and mixes ordinary Lennard–Jones parameters on demand. An earlier all-pairs metadata representation required $32N^2$ bytes and imposed a practical allocation ceiling; the sparse exception representation removes that quadratic metadata cost. OBC2/ACE is evaluated in three kernel passes for Born radii, coordinate derivatives, and solvent/nonpolar energy and force accumulation. Optional X–H constraints are graph-coloured so non-overlapping SHAKE/RATTLE corrections can execute concurrently [13, 14]. Seeded Langevin dynamics supports 1 fs flexible and 2 fs constrained integration.

Pocket relaxation uses a bounded steepest-descent procedure with zero inverse mass on fixed receptor atoms; the ligand and protein side chains in the automatically selected 5 Å pocket move, while the protein backbone and outer atoms remain fixed. Current operating limits are listed in Table 2.

Table 2: Operating limits in the current direct WebGPU worker.

Setting	Current value
Principal force workgroup	64 atomwise calculations
Time step	1 fs flexible; 2 fs with X–H constraints
Constraint solve	Four graph-coloured sweeps by default; at most 32
Minimization	750 iterations by default; at most 2,000; each displacement capped at 0.001 nm
Direct dynamics request	At most 5,000 steps; longer requests use the ensemble route
Validation cutoff	1.0 nm cutoff; 0.2 nm skin; rebuild every 20 steps

At the interactive system sizes tested here, a conventional Verlet neighbour list did not improve performance. With a 0.2 nm skin rebuilt every 20 steps, 5,000 constrained OBC2 Trp-cage steps were 46% slower and 1,000 ubiquitin steps were 17% slower than full-range nonbonded evaluation on an Apple M1 Pro. The cutoff route is therefore retained for validation but is not exposed in the product controls.

GPU benefit depends on amortizing dispatch and data movement. In the earlier browser benchmark, a single Trp-cage energy/force evaluation required 25.5 ms in WebGPU and 5.7 ms in the bundled OpenMM WebAssembly Reference implementation. Across 1,000 dynamics steps, WebGPU reached 214.9 versus 102.8 ns/day for 304-atom Trp-cage and 65.18 versus 7.11 ns/day for 1,231-atom ubiquitin. These measurements use warmed, nonperiodic, all-pairs workloads. The direct path does not currently implement periodic boundaries, particle-mesh Ewald electrostatics, explicit solvent, barostats, virtual sites, or arbitrary OpenMM custom forces.

A.1.3 Replica-parallel WebGPU

The replica engine targets throughput across independent copies of a shared topology. One replica maps to one GPU workgroup, in contrast to the atomwise decomposition used by the direct engine. The memory layout and fixed-point accumulation strategy draw on STORMM's stacked-system design, while the browser implementation is independent WGSL code [10]. Replicas share topology and parameters but do not interact.

Concurrent accumulation uses paired 32-bit words to represent signed 64-bit fixed-point values because WGSL exposes 32-bit integer atomics. The implementation uses 2^{22} counts per kcal/mol for energies, 2^{18} counts per kcal/mol/Å for forces, and 2^{32} counts per Å for coordinates. This replaced 32-bit accumulators that overflowed silently in highly clashed configurations and produced bit-for-bit replay for identical seeds on the tested adapter.

Integration uses velocity Verlet, seeded Langevin thermalization, and SHAKE/RATTLE. Constraint projection is serial within a replica while replicas execute in parallel; the default relative tolerance is 10^{-5} with at most 32 iterations. Commands contain at most 50 steps. Persistent status words convert non-finite values, accumulator overflow, coordinate clamps, and failed constraints into terminal errors. Current limits are 512 atoms per replica, all-pairs nonbonded evaluation, and identical atom count, topology, and parameters across replicas.

Force evaluation uses a separate 32-bit floating-point coordinate working copy. Translating a test system 500 Å from the origin changed energies by approximately 3×10^{-4} relative, identifying the working coordinate representation rather than the fixed-point store as the remaining translation-sensitivity boundary. Eliminating this source of error requires recentering before force evaluation or a wider working representation.

Replica throughput also crosses over with workload size. In the earlier warm comparison against bundled OpenMM WebAssembly Reference, that reference was 13.3-fold faster for one C16 molecule and 3.5-fold faster for one 27-water system. WebGPU was 10.3-fold faster for 1,024 C16 replicas and 24.1-fold faster for 256 water replicas. These measurements compare aggregate replica throughput; the two engines do not use identical integrators.

A separate production-worker benchmark compares browser WebGPU directly with independently constructed native OpenMM Systems on Apple M1 Pro and NVIDIA L4, without relying on transitive agreement through WebAssembly. The direct worker passes all 47 fixed-f32-input cases and 42 of 47 original-input cases; these are distinct precision gates. The STORMM-style worker passes all 22 supported original-input cases, retaining 25 unsupported cases explicitly. These results concern implementation agreement, not experimental force-field accuracy. The versioned benchmark supplement records every energy, Cartesian force vector, tolerance, timing sample, and input identity. Its browser timings include fresh production-job overhead, whereas native GPU baselines use resident

Contexts; the column ratios are not matched-kernel speedups or the earlier multi-replica throughput comparison.

A.2 Reference-driven implementation and validation

The numerical implementation is developed against a shared executable specification. OpenMM and the browser kernels receive the same molecular graph, atom ordering, coordinates, parameters, units, exceptions, constraints, and named energy terms. This enables term-resolved differential testing rather than comparison of independently prepared end states. The implementation loop is:

1. freeze the molecular graph, atom ordering, coordinates, force-field parameters, and units used by the reference calculation;
2. implement one energy/force term and its memory representation at a time;
3. evaluate the identical numerical input with OpenMM;
4. compare named energy components and Cartesian force components;
5. localize the first discrepancy, correct it, and retain the minimal reproducing case as a regression test; and
6. execute the resulting WebGPU and WebAssembly paths in the browser to include worker, adapter, serialization, and UI integration boundaries.

Coding agents operate inside this harness by modifying implementation code, tests, and adapters. The physical-model equivalence criteria—including atom order, solvent model, cutoffs, constraints, units, charge provenance, and acceptable numerical tolerances—are specified explicitly and remain prerequisites for interpreting any backend comparison.

Several observed failures led directly to architectural changes (Table 3).

Table 3: Observed failures that changed the implementation.

Observation	Design change	Retained check
Pair metadata grew as $32N^2$	Store only exceptional pairs; mix ordinary pairs on demand	Large-system allocation and parity tests
32-bit force/energy sums overflowed	Paired-word 64-bit fixed-point accumulation and fault flags	Clashed system at approximately 2 million kcal/mol
Coordinates wrapped beyond 128 Å	64-bit fixed-point coordinate store plus floating-point working copy	Whole-system translation to 500 Å
Standard cutoff reduced performance	Retain for validation; omit from product controls	Fixed Trp-cage/ubiquitin timing protocols
Apparent engine mismatch	Verify atom order, input hashes, and solvent mode before numerical debugging	Native/WebAssembly parity and matched-solvent comparisons

Regression coverage is organized around the interfaces where failures were observed: equation-level fixtures for bonded and nonbonded terms; representation tests for clashes, constraints, overflow, and translated coordinates; backend comparisons on identical prepared inputs; protein fixtures that exercise special pairs and worker transfer; browser tests for adapter support and worker recovery; and fixed performance protocols that retain slower but numerically valid designs. Fixed random seeds and stable atom identifiers make failing cases repeatable. The replica engine terminates on fixed-point overflow or unconverged constraints; the direct engine records final constraint error for comparison against an acceptance threshold.

The demonstrated portability boundary is fixed-topology, nonperiodic molecular mechanics with regular array state and a small number of synchronization points. WebGPU is effective when work decomposes into local or pairwise operations and repeated evaluations amortize dispatch. Algorithms requiring 64-bit floating-point arithmetic, frequent device-wide synchronization, rapidly changing graphs, large adaptive sparse structures, or mature FFT/sparse-linear-algebra libraries require a different decomposition under current WGSL [22]. Particle-mesh Ewald electrostatics, heterogeneous reactive systems, and some long-timescale production protocols are therefore outside the demonstrated browser-native boundary. For those workloads, the architecture supports escalation to a WebAssembly reference, hybrid local path, or external specialist engine while preserving the originating molecular state and protocol provenance.

A.3 Workflow orchestration and local execution boundary

Molarium distinguishes local calculation from offline operation. In normal connected mode, supported numerical calculations execute on the user's device, while explicit PDB/CCD retrieval or sequence-alignment searches contact the services identified in the interface. Local Lab serves a reviewed, versioned checkout from localhost under a restrictive Content Security Policy. External structure and alignment controls are disabled, and an egress-canary test fails if a synthetic private-data marker reaches an external request. A hosted application can change the assets served on a subsequent visit; the stronger reproducibility and privacy boundary is a reviewed checkout with locally pinned assets.

Current workflow surfaces include:

- **Local small-molecule design.** Synchronized 2D/3D construction, graph validation, RDKit or direct-WebGPU calculations, aligned trajectory/energy inspection, and export. OpenMM/WebAssembly is used for internal validation and fixed-scaffold operations rather than as a selectable Simulate-panel method.
- **Protein–ligand pose propagation.** A trusted pose is captured with stable atom lineage. Constrained propagation searches edited groups and eligible edit-associated torsions against a fixed receptor, retaining the protected core and enforcing selected required-contact checks. Pocket relaxation moves the ligand and protein side chains in a 5 Å pocket while fixing the backbone. Experimental induced-fit relaxation permits whole-residue motion within 6 Å, including backbone, and eligible ligand/water motion, but fixes ligand atoms protected by registered retention rules. Neither minimization lane enforces the docking hydrogen-bond restraints; contacts must be inspected afterward. Discrete side-chain rotamer enumeration is a separate operation, not an automatic consequence of minimization.
- **Agent-operated design.** Agents invoke the versioned Chemist Actions API rather than arbitrary page functions. The API exposes the same bounded state-changing verbs available to a chemist—including select, edit, search, apply, relax, and undo—plus read-only inspection. Requests execute serially and record sequence, timestamps, arguments, status, and duration.

Two extension paths are represented in the architecture but are not complete product workflows. A local-to-specialist path can export a fingerprinted checkpoint and explicit protocol for periodic solvent, long trajectories, free energy, or electronic-structure calculations, with returned results attached as evidence to the originating branch; transport, scheduling, signatures, and trust policy are not implemented. A collaborative review model can branch by chemotype, protonation state, pose, or author and record a selected branch with explicit parentage; coordinate averaging is not used as a merge operation.

A.4 Content-addressed molecular and decision provenance

Molarium separates operational, numerical, and decision provenance. A molecular snapshot records one explicit state. Content-addressed commits link that state to parent states, action scripts, hypotheses, and evidence. Branches preserve alternative design routes, and decisions record which route progressed, stopped, or was deferred. These semantics are implemented as a custom content-addressed ledger rather than a Git repository or a general structural merge algorithm.

The live application records public actions in memory; clearing the scene clears that transient audit. The Design History interface and public `campaign.*` actions can create a local campaign, commit graph-and-coordinate snapshots and completed actions, create and restore branches, record decisions, merge branch histories, and verify the append-only record. Campaign JSON and the active branch are persisted in browser IndexedDB. Explicit export is needed for portable handoff. Attached action scripts cover completed public operations, not every transient interaction or the complete workbench state.

Canonical JSON and SHA-256 fingerprints bind records to exact serialized representations. The `molarium.design-campaign/v1` schema stores snapshots, commits, branches, events, and decisions. Each commit identifies its selected snapshot, parent commit(s), optional action script, hypotheses, evidence, tags, and branch. Events are append-only and contain the previous event fingerprint, actor class (human, agent, system, or import), and source. Supported decision states are `progressed`, `not-progressed`, `deferred`, `failed`, `duplicate`, `superseded`, and `archived`. Finalization appends a completion event and hashes the campaign; subsequent record changes fail verification. Hashes provide alteration detection for the recorded representation, not authorship, chemical equivalence, or scientific validity.

Table 4: Separation of operational, numerical, and decision provenance.

Record	Question	Contents
Chemist Actions audit	What was requested?	Public action, arguments, timing, status, and compact after-state
Calculation record	What numerical protocol ran?	Inputs, method, parameters, outputs, and hash-linked events
Design campaign	Why did a state advance or stop?	Snapshots, branches, parents, hypotheses, evidence, and decisions

Registered design routes use a separate `molarium.registered-design-route/v1` schema that specifies the coordinate boundary, hash-pinned starting state, and ordered graph edits. The SOS1 route uses this schema. Route actions are `designRoute.load`, `designRoute.applyStep`, and `designRoute.inspect`; campaign-ledger operations remain separate. Cross-schema validation rejects a route presented as a campaign ledger or a campaign ledger presented as a route. Frozen predictions made before the schema split retain their original input hash, with migration records storing both hashes and accepting only the schema-identifier substitution that reconstructs the frozen bytes.

A decision-to-state linkage has the following form; full identifiers are SHA-256-derived values rather than the abbreviated labels shown here.

```
{
  "moleculeCommit": {
    "snapshotId": "snapshot:<selected-child>",
    "parents": ["commit:<parent>"],
    "actionScriptId": "script:<designer-moves>",
    "hypothesisIds": ["event:<pocket-must-open>"]
  },
  "decisionEvent": {
    "kind": "decision.recorded",
    "actorId": "chemist.01",
    "payload": {
      "targetCommitId": "commit:<selected-child>",
      "disposition": "progressed",
      "reasonCodes": ["contact-loss"],
      "rationale": "Phe-out removes growth-induced clashes.",
      "evidenceIds": ["event:<rotamer-search>"]
    }
  }
}
```

Executable replays use `molarium.chemist-action-script/v1`. Scripts contain ordered calls to the allow-listed Chemist Actions API rather than arbitrary code or direct coordinate replacement. Designer Moves JSON files must be smaller than 2 MB. Ordinary API requests are capped at 8 MiB for ASCII JSON, twelve nested levels, and 2,048 JSON nodes. Loading a multi-action script permits 32,768 nodes while retaining the depth and byte limits; each constituent action uses the ordinary request limits. Serialized full-system campaign imports have a separate 32 MiB allowance. Actions can name returned stable identifiers for reuse by later steps. The runner executes one action at a time through `api.execute({action,args})`, derives a repeatable request ID from the script fingerprint, and stops on the first failure. Replay results use `molarium.chemist-action-replay/v1` and record the source fingerprint and each step outcome. Presentation actions can alter focus or styling but cannot change molecular graph or coordinates except through the same allow-listed state-changing actions.

The SOS1 replay is identified by a stable public route and by exact release and content fingerprints. A human-readable URL can resolve to a newer release, whereas the release commit, route-file fingerprint, and canonical action-script fingerprint identify the reviewed bytes and normalized JSON content. The live workflow can capture a starting state, record exploration actions, apply and relax selected states, and export action and numerical records. Formal campaign commits and decision events are represented by the ledger layer described above.

A.5 SOS1: molecular design as an executable protocol

The SOS1 example represents a molecular-design procedure as an ordered program of graph edits, geometric instructions, interaction hypotheses, permitted atomic motions, candidate evaluations, and selection rules. The sequence follows the analogue series reported by Hillig and colleagues [30]. The program records which quantities are specified by the designer and which are determined by calculation. It is an executable representation of a design hypothesis, not a reconstruction of the original medicinal chemists' recorded decisions.

A.5.1 Inputs and preparation

The starting coordinates are the 5OVE/AXE complex. The subsequent molecular graphs are AWT, AWZ, AWW, and AXH (BAY-293). The later crystal series was inspected to infer the AWW arm direction and intended Tyr884 contact; those crystal coordinates were not imported into the trajectory or used as coordinate targets. The procedure is therefore reference-informed rather than blinded.

Preparation uses pH 7.4, automatic histidine assignment, CCD ligand topology, missing-heavy-atom repair, capped gaps, and retained source waters. Parameterization assigns whole-system OpenFF Sage 2.1.0 parameters and deterministic Gasteiger charges without moving coordinates. The AWT, AWZ, AWW, and AXH systems contain 7,929, 7,933, 7,935, and 7,937 particles, respectively. The accompanying run records retain the assigned atomic charges and numerical parameters for each state.

The registered graph edits preserve atom lineage through element- and bond-order-exact mappings of conserved subgraphs (Table 5). Each new analogue starts from the preceding computed state. Molecular identity and pose-transfer rules are inputs to the operation; the later experimental pose is not.

Table 5: Heavy-atom lineage across the supplied SOS1 analogue graphs.

Transition	Retained	Deleted	Added	Candidate mappings
AXE–AWT	23	4	6	4
AWT–AWZ	19	10	12	1
AWZ–AWW	16	15	15	1
AWW–AXH	15	16	17	1

A.5.2 Design procedure

The AWW geometry instruction separates ligand placement from receptor response. The primary C12–C15 rotation is specified as +150°. The upstream N7–C12 rotation is restricted to 0–60°; two downstream torsions and donor-hydrogen orientation are searched against the declared Tyr884 backbone-carbonyl contact. Atom selectors resolve atoms in the current structure. Coordinates outside the declared moving branch remain fixed. During ligand placement, only the six movable Phe890 ring atoms are permitted to overlap the growing ligand; Phe890 Cβ, other receptor atoms, and retained waters are not exempt from clash checks.

Once the ligand pose is recorded, it is locked. Receptor sampling changes only Phe890 while the ligand, backbone, other receptor atoms, and source waters remain fixed. Tyr884 supplies a backbone-carbonyl contact; the procedure does not model a Tyr884 side-chain flip or backbone transition. This defines the calculation being tested: the receptor response to a specified ligand pose.

A.5.3 Pseudocode for the complete movie

Listing 1 condenses the complete design sequence from the API JSON into pseudocode. Operation names follow the public API, while chemical selectors and arguments are abbreviated and repeated trials become loops. The checkpoint annotations identify the seven saved states shown in the movie; they are not additional API calls. Repeated inspections, workspace changes, and session-resumption calls are omitted. The listing is a reading guide, not a separately executable script. A `#` introduces an explanatory comment; unprefixed lines specify operations or checks, including `require` and `inspect`. Indented argument continuations belong to the preceding instruction.

Listing 1: The whole SOS1 movie as pseudocode condensed from the public API action script. Angles are in degrees; distances are in angstroms unless stated otherwise.

```
designRoute.load("sos1-hit-only")                # 5OVE / AXE
protein.prepare(pH=7.4, histidine=auto, ligand=CCD,
                repairMissingHeavy=true, gaps=cap, waters=retain)
pose.captureReference(mode=propagate)            # checkpoint 1: hit
for step in [scaffold-rewrite, fragment-merge]:   # AWT, then AWZ
    designRoute.applyStep(step)
    pose.refine(searchChains=8, execution=serial, featureSeeding=v5)
    require complete coverage and feasible pose with retained core
    pose.apply(index=0)
    protein.parameterize()                       # no coordinate motion
    optimization.run(method=induced-fit-webgpu)
    require accepted relaxation, valence, and registered pose retention
    # checkpoints 2 and 3: relaxed AWT and AWZ
    pose.captureReference(mode=propagate)
protein.parameterize()
designRoute.applyStep(open-phe890-pocket)         # checkpoint 4: AWW graph
protein.parameterize(); pose.captureReference(mode=propagate)
pose.addContact(N7 donor -> Asn879.OD1)
pose.addContact(OX3 donor -> Tyr884.backboneO)
geometry.alignBranchToContact(target=Tyr884 contact, solution=best-directional,
    primary=C12--C15:+150, upstream=N7--C12:0..60,
    coupled=[CX4--CX5, CX15--CX16], donorHydrogen=search,
    allowedResponseAtoms=Phe890.[CG,CD1,CD2,CE1,CE2,CZ])
require no external coordinate targets, no internal severe clashes,
        and no overlaps outside the allowed Phe890 response atoms
pose.setDesignerLigandPoseFixed(true)            # checkpoint 5: ligand intent
protein.parameterize(); pose.setDesignerLigandPoseFixed(true)
for candidate in pose.enumerateSidechainRotamers(Phe890, maximum=64):
    pose.applySidechainRotamer(candidate)         # 13 states, including input
    inspect and retain ligand/pocket coordinates and severe-clash count
    calculation.run(job=energy, method=openmm,
                    solvent=OBC2, cutoff=1.0 nm, constraints=none)
    retain energy; require ligand coordinates unchanged
    history.undo(); verify restored ligand       # re-enumerate before next trial
# Source-run rule: lowest finite energy among zero-severe-clash states.
# The replay reapplies the recorded choice; publication verification
# requires it still to satisfy that rule under the fresh calculations.
pose.applySidechainRotamer(chi=[-180,90])          # checkpoint 6: Phe890 response
protein.parameterize(); pose.setDesignerLigandPoseFixed(false)
pose.captureReference(mode=propagate)
designRoute.applyStep(finish-bay-293)              # AXH graph; retain distal
                                                 # seven-atom feature, RMSD <=2.25
pose.refine(searchChains=8, execution=serial, featureSeeding=v5)
require complete coverage, feasible pose, retained core and required feature
pose.apply(index=0); protein.parameterize()
optimization.run(method=induced-fit-webgpu)
require accepted relaxation, fixed-atom, valence, and feature-retention checks
# checkpoint 7: BAY-293; inspect and save the resulting ligand and pocket
```

The complete executable action script contains 159 executable public actions, with the full selectors, arguments, and result expectations. Its runner stops on a failed action or expectation. Candidate coordinates, energies, and rejected alternatives remain in the audit. The accompanying human account explains the choices and outcomes at each checkpoint, following the paired human/agent approach used in Appendix B.

A.5.4 Human account: decisions and outcomes

At checkpoint 1, the prepared 5OVE/AXE hit defines the coordinate starting point. The later analogue graphs and the crystal-informed design hypothesis are supplied information, not discoveries made by the program. Retaining the source waters and choosing a preparation and charge-assignment protocol are modeling assumptions carried through the sequence.

Checkpoints 2 and 3 ask whether the scaffold rewrite to AWT and fragment merge to AWZ can be accommodated while retaining the mapped core. Each edit first transfers the preceding pose, then searches eight pose-propagation chains. The selected feasible pose undergoes coupled ligand–pocket relaxation. Both steps passed the recorded coverage, core-retention, valence, and relaxation checks. This establishes usable starting states for the next edit, not the experimental accuracy of either intermediate. Constrained propagation preserves declared pose features during search; induced-fit relaxation subsequently adjusts permitted ligand and pocket coordinates locally. The latter is not a search over discrete receptor rotamers.

Checkpoint 4 separates molecular identity from placement: installing the AWW graph does not itself specify the intended arm direction. At checkpoint 5, the designer supplies that direction and declares the Asn879 and Tyr884 contacts. The bounded search places the arm against the Tyr884 backbone carbonyl; declaring the Asn879 contact alone does not establish that it is satisfied. The selected pose meets the Tyr884 geometry requirement, with a donor–acceptor distance of 2.829 Å and an angle of 155.288°. It has no severe ligand-internal clashes or overlaps outside the permitted Phe890 ring atoms, but it still overlaps that ring. This checkpoint therefore records a ligand hypothesis that requires a receptor response, not an accepted final complex. No Tyr884 side-chain flip is involved.

Checkpoint 6 asks whether Phe890 can accommodate that exact ligand pose. Locking the ligand prevents the calculation from resolving the problem by moving the proposed ligand instead. Of 13 enumerated receptor states, including the unchanged input, two have zero severe clashes; the unchanged input has 11. All 13 receive fixed-coordinate, full-system OpenMM energy evaluations with OBC2, a 1.0 nm cutoff, and no constraints. The lower-energy clash-free state has library angles $(\chi_1, \chi_2) = (-180°, 90°)$. Thus discrete sampling resolves the steric conflict within the supplied candidate set while preserving the ligand and its Tyr884 contact. It does not establish the unique receptor conformation or a dynamical flipping pathway. Relative to 5OVH, the arm torsion differs by 0.341° and Phe890 χ_1 and symmetry-aware χ_2 by 4.236° and 13.672°.

Checkpoint 7 tests a different instruction: change the attachment to the BAY-293 graph while retaining a seven-atom distal spatial feature. The AWW coordinate lock is released; retaining a feature allows bounded movement rather than requiring identical coordinates. Of eight pose-search candidates, three are feasible: four others violate feature retention and one fails the physical-feasibility check. The selected feasible pose then undergoes induced-fit relaxation and passes the fixed-atom, valence, and feature-retention checks. The final 32-heavy-atom ligand has no severe internal or protein clashes. Its distal feature moves by 1.529 Å RMSD and 1.289 Å in centroid position, within the registered 2.25 Å RMSD tolerance. This is approximate spatial retention, not exact reproduction; Phe890 side-chain RMSD to 5OVI is 0.928 Å.

The states were saved before numerical comparison with the later crystals. The final ligand comparisons in Table 6 use pocket Cα alignment and graph-symmetry minimization, without independently fitting the ligand. BAY-293 shows a larger deviation than AWW. These measurements describe the outcome of the stated, reference-informed design procedure; they do not demonstrate blinded pose prediction or affinity improvement.

Table 6: Structural comparison of the saved SOS1 design states. Ligand RMSD is receptor-aligned and graph-symmetry-minimized.

Checkpoint	Crystal comparison	Ligand RMSD (Å)
AWW ligand hypothesis and Phe890 response	5OVH	0.851
BAY-293 continuation	5OVI	1.880

A.5.5 Recomputation and inspection

The release provides three representations of the same procedure. The recomputable replay executes public actions and generates new coordinates and energies; these may differ from the saved run. Its publication check requires the recorded Phe890 choice to remain the lowest-energy clash-free candidate under the fresh calculations. The

precomputed replay imports seven exact checkpoint campaigns without recalculation. The MP4 records navigation through those checkpoints, with five one-second popups showing the recorded calculation-status text. Its 62.75 s duration is presentation time, not calculation time. The release record identifies the action script and checkpoint files; the movie manifest identifies the rendered sequence.

This representation makes a molecular-design procedure explicit at three levels: the supplied chemical hypothesis, the operations and selection rules used to evaluate it, and the resulting molecular states. Changing a contact, allowed motion, or selection rule defines a different procedure whose results must be recomputed. The saved coordinates document the outcome; the API program documents how that outcome was obtained.

APPENDIX B
Working with Molarium: scientific workflows for humans and agents

Authorship note. This appendix shares the AI-drafting and limited-human-editing disclosure at the start of Appendix A.

Molarium is designed so that a scientist working through the graphical interface and an agent operating through the Chemist Actions API act on the same molecular state and encounter the same scientific boundaries. This appendix therefore describes Molarium by **scientific task**, not by software module.

Each workflow begins with the question a scientist is trying to answer. The human procedure explains how to perform the task and what to inspect before accepting the result. The accompanying **agent skill contract** expresses the same procedure as preconditions, permitted actions, evidence, success criteria, and stopping conditions. The two views are intentionally paired: a procedure should be understandable by a scientist and sufficiently explicit that an agent cannot silently change its meaning.

Implementation basis. This appendix documents the browser workflows and public API, distinguishing established operations from experimental capabilities. Repository-only and proposed functionality is not presented as an operating instruction. User-facing workspace names are View, Design, and Simulate; the serialized values remain `view`, `build`, and `run` so existing action scripts remain replayable.

B.1 One molecular state, two interfaces

Status. Current browser surface.

The question. What changes the molecule, and what merely changes how I am looking at it?

Molarium separates the authoritative molecular state from the interface used to manipulate it. A scientist can work in **View**, **Design**, and **Simulate**; an agent can invoke the same guarded scientific operations through the versioned Chemist Actions API. Both routes act on the same graph, coordinates, selections, calculation records, and pending chemistry transaction. Browser-native transport—choosing a local file, writing to the clipboard, downloading a file, or entering fullscreen—remains outside the molecular API; the state-changing operation before or after that boundary has a named action.

Rotating the camera, changing a representation, focusing a component, or showing contacts changes only the presentation. Loading a structure, applying a protonation state, finishing a chemistry transaction, accepting a docking candidate, running a coordinate-producing Simulate job, or selecting or playing a saved calculation frame changes the current coordinates or graph. Refined-pose and side-chain alternatives remain candidates until explicitly applied.

The common operating rule. Inspect, act, verify. Generate alternatives freely, but change the authoritative molecular state deliberately. A valid operation is not automatically a scientifically appropriate one; acceptance still depends on the question being asked.

Starting from a structure

Start with `?blank` when a reproducible workflow should begin from an empty canvas. Otherwise the application loads its bundled launch molecule. A structure can be pasted as XYZ or PDB text, uploaded as XYZ or PDB, entered as a SMILES string or PDB identifier in connected mode, or selected from the prepared examples. The current upload router should not be relied on for ordinary MOL-file ingestion, even though a V2000 parser exists elsewhere in the application.

After loading, inspect atom count, formula, formal charge, connected components, graph connectivity, and the 3D structure. For a selected small-molecule component, compare the synchronized 2D depiction with the 3D graph. For PDB input, remember that only the first model is loaded and alternate locations are resolved by the implemented occupancy policy. For XYZ input, bonds are inferred from proximity and should be checked rather than assumed.

State and persistence

Loading a new molecule resets incompatible calculation, docking, selection, and preparation state. The working session is in memory: Clear, page reload, or script import can discard unsaved work. Share copies a URL rather

than serializing the current molecule, and Save produces a PNG rather than a reusable molecular record. Export the scientific record needed by the next step before replacing or clearing the scene.

Agent skill contract: `operate_molecular_state`

Contract element	Requirement
Goal	Load and inspect a starting structure without confusing a visual change, candidate, or imported approximation with an accepted molecular state.
Preconditions	A supported input or prepared example is available; connected-mode requirements are known.
Inputs	Input type and contents; intended component; optional network policy; requested inspection fields.
Procedure	Start blank when reproducibility requires it; load through a supported route; inspect graph, components, charge, and coordinates; report ambiguities before any mutation.
State mutation	A successful load replaces the current molecule and resets dependent state. View and selection operations do not alter graph chemistry.
Evidence	Source identity or input hash where available; atom/component summary; explicit warnings about inferred bonds, alternate locations, or network retrieval.
Success	The loaded graph and coordinates correspond to the intended starting object and all material ambiguities have been surfaced.
On failure	Leave the prior scene in place where the interface does so; report the parse, fetch, or format error; do not reinterpret an unsupported file silently.
Boundary	No autosave or complete-session archive; an image or share link is not a reusable molecular state.

Structure text, identifiers, and prepared examples have explicit loading actions. A local file picker only supplies bytes to `session.loadStructure`; it is not an alternate molecular loader:

```
await api.execute({action:"session.loadStructure", args:{
  format:"pdb", content:pdbText, name:"starting complex"}})
// alternatives:
await api.execute({action:"session.loadIdentifier",
  args:{value:"5OVE", kind:"pdb"}})
await api.execute({action:"session.loadFixture",
  args:{fixtureId:"trp-cage"}})
await api.execute({action:"session.inspect",
  args:{scope:"all", includeCoordinates:false, maximumAtoms:500}})
await api.execute({action:"view.setDisplay",
  args:{representation:"both", showHydrogens:false,
        showPocketAtoms:true, showInteractions:true}})
```

Registered prospective routes and public structure stories retain their narrower guarded loaders (`designRoute.load` and `structureStory.load`).

B.2 Prepare a protein for local molecular design

Status. Experimental current surface.

The question. I have an experimental protein structure. What must happen before I can safely edit a ligand or run a local calculation?

A deposited PDB structure is not automatically a simulation-ready system. Atoms may be missing, alternate locations may have been chosen, protonation can be ambiguous, crystallographic waters may or may not matter, and ligands or unusual residues may exceed the supported parameterization. Molarium's preparation workflow is deliberately conservative: it makes a bounded set of assumptions explicit, performs repairs it knows how to perform, and stops when the structure exceeds those capabilities.

Human workflow

1. Load the PDB structure and inspect the protein, ligand, waters, other components, and the region around the intended binding site before preparing anything.
2. Open **Protein Preparation**. Choose pH, histidine policy, supported missing-heavy-atom repair, ligand treatment, water treatment, and gap policy.

3. Run **Prepare structure**. Molarium constructs an internal preview that inventories residues, missing atoms, sequence gaps, ligands, waters, clashes, and invalid coordinates. Within scope, it can retrieve CCD information, reconstruct supported missing side-chain atoms, add standard hydrogens and terminal oxygen, choose histidine states, and scan polar-hydrogen orientations.
4. An unresolved blocker stops the operation and exposes the preview for review. A blocker-free preview is immediately parameterized and applied; there is no normal second Apply step.
5. Inspect every applied change, warning, and attached audit. Download the preparation report if the decisions must survive the browser session.

What should I inspect?

- Ligand identity, connectivity, completeness, and whether it should be retained or excluded.
- Histidine and other local protonation choices near the ligand or catalytic region.
- Crystallographic waters that bridge interactions or define the local geometry.
- Missing atoms, chain breaks, unresolved regions, reconstructed geometry, and severe clashes near the region of interest.
- Whether the resulting force-field and charge assumptions match the scientific question.

Acceptance is narrower than success. A clean preview means Molarium completed its declared preparation procedure. It does not certify a production molecular-dynamics system. The implementation explicitly records `readyForProductionSimulation: false`.

Preparation blocks unsupported residues, unsupported missing atoms, incomplete retained ligands, disallowed gaps, invalid coordinates, and severe clashes. It does not currently construct missing loops or membranes, model general metal coordination, parameterize covalent ligands, or build periodic explicit-solvent boxes. Arbitrary proteins use an experimental Sage/Gasteiger route.

Agent skill contract: `prepare_protein`

Contract element	Requirement
Goal	Convert the loaded PDB into an explicitly reviewed molecular state suitable for a supported local Molarium calculation.
Preconditions	A PDB-derived structure is loaded; the intended downstream use is known; ligand and water roles can be reviewed.
Inputs	pH; histidine policy; missing-heavy-atom repair policy; ligand policy; water policy; gap policy.
Procedure	Inspect the starting structure; run preparation; review any blocking preview or the applied result; export the report when persistence matters; stop on unresolved blockers.
State mutation	Prepared graph and coordinates replace the prior state; preparation audit and parameterization are attached; one undo snapshot is created.
Evidence	Options, detected issues, repairs, warnings and blockers, preparation report, and resulting parameterization identity.
Success	The declared procedure completes without unresolved blockers and its scope matches the intended local calculation.
Failure behavior	Report the blocker and stop. Do not manufacture missing atoms, parameters, loop geometry, or unsupported chemistry.
Escalate when	The problem requires loops, membranes, metal coordination, covalent chemistry, periodic solvent, or another unsupported preparation capability.
Does not establish	Production simulation readiness or physical accuracy of the chosen downstream model.

Representative public actions are:

```
await api.execute({action:"protein.prepare", args:{
  pH:7.4, histidine:"auto", repairMissingHeavy:true,
  ligandPolicy:"ccd", waterPolicy:"crucial", gapPolicy:"block"}})
await api.execute({action:"session.inspect",
  args:{scope:"all", includeCoordinates:false, maximumAtoms:500}})
```

The preparation action already parameterizes and applies a blocker-free result. `protein.parameterize` is a separate alternative for an already prepared or subsequently edited complex, not the next mandatory call. The exact

argument vocabulary returned by `api.describe()` is the runtime authority; this sequence illustrates the contract rather than replacing that manifest.

B.3 Choose the protonation state you are designing

Status. Current ligand workflow; protein choices within experimental preparation.

The question. What charged and protonated species does the molecular graph actually represent?

Protonation is part of chemical identity, not a cosmetic display choice. It changes formal charge, hydrogen count, hydrogen-bonding patterns, parameterization, and often the meaning of every calculation that follows. Molarium exposes two related but distinct workflows: ligand-state enumeration through RDKit, and protein protonation decisions inside the bounded PDB-preparation workflow.

Ligand workflow

1. Load the ligand from a SMILES string or select the ligand component whose identity you intend to change.
2. Choose a pH from 0 to 14 and enumerate candidate states. RDKit applies a versioned Dimorphite-style site table, edits formal charges and hydrogens, sanitizes each result, and returns an ordered list with estimated weights.
3. Inspect each candidate as chemistry: total charge, changed atoms, tautomeric or aromatic consequences, and compatibility with known assay conditions or the bound environment.
4. Apply one state deliberately. Molarium re-embeds the chosen species, replaces the graph and coordinates, and invalidates state that depended on the former topology.

Protein workflow

For a PDB structure, set the pH and histidine policy during preparation. The implemented policies include automatic selection or explicit HID, HIE, and HIP choices, followed by inspection of polar-hydrogen orientations. Treat residues near a ligand, metal, catalytic center, or salt-bridge network as scientific decisions rather than defaults.

What the reported ligand weights mean. They are independent Henderson-Hasselbalch estimates from the implemented site model. They are not microscopic pKa predictions, coupled-site populations, or evidence for the state selected by a protein binding site.

Agent skill contract: `select_protonation_state`

Contract element	Requirement
Goal	Make the intended protonation and formal-charge state explicit before editing or calculation.
Preconditions	A valid ligand or PDB structure is loaded; the relevant pH or experimental condition is known or stated as an assumption.
Inputs	pH; candidate-state selection; for proteins, histidine policy and local residues requiring review.
Procedure	Enumerate or preview; compare candidates; inspect charge and local interactions; apply exactly one accepted state; re-inspect the resulting graph.
State mutation	Applying a ligand state replaces graph and coordinates. Protein protonation changes are committed through preparation. Prior parameterization becomes obsolete.
Evidence	pH, enumeration or preparation output, chosen state, total charge, changed sites, and stated rationale when more than one state is plausible.
Success	The authoritative graph explicitly represents the intended state and downstream methods will see the corresponding atoms and charges.
On failure	Report invalid pH, sanitization, embedding, or worker errors; do not partially apply or choose a different state merely because it runs.
Boundary	Enumeration weights do not resolve coupled microscopic states or protein-induced pKa shifts.

Ligand-state enumeration and application use the same bounded worker and graph replacement as the visible pH controls:

```
await api.execute({action:"ligand.enumerateProtonation",
  args:{pH:7.4, maximumStates:16}})
await api.execute({action:"ligand.applyProtonation", args:{index:0}})
```

```
await api.execute({action:"session.inspect",
  args:{scope:"ligand", includeCoordinates:false}})
```

Enumeration does not select a chemically correct state. The chosen index remains a reviewable scientific decision; protein protonation is carried separately within `protein.prepare`.

B.4 Edit atoms, bonds, charges, and stereochemistry in 3D

Status. Current browser surface.

The question. How do I change the chemistry without corrupting the molecular graph or losing the ability to undo the operation?

The synchronized 2D and 3D views are two ways of selecting the same graph. Chemistry-changing actions are grouped into a transaction so that a related set of edits can be validated and committed as one scientific step. Protein covalent atoms are protected from ordinary ligand editing.

Human workflow

1. Enter **Design** and select the relevant atom or bond in either view. Confirm the stable atom identity and local environment before changing it.
2. Apply the intended element replacement, bond order or aromaticity change, formal charge, explicit hydrogen, or stereochemical operation. Related operations remain in one pending chemistry transaction.
3. For coordinate-only geometry changes, select a connected path of two, three, or four atoms to set a distance, angle, or dihedral. Molarium moves the connected side defined by the graph and blocks ambiguous ring cuts.
4. Choose **Finish chemistry**. RDKit sanitizes the graph, hydrogens are reconciled, eligible local coordinates are polished, reference contacts are remapped where applicable, obsolete parameterization is invalidated, and a single undo snapshot is created.
5. If validation fails, correct the still-pending transaction or choose **Discard** to restore the pre-edit state. Use Undo/Redo only after a transaction has been committed.

What should I inspect?

- Valence, formal charge, aromaticity, hydrogen count, stereochemistry, and the intended bond topology.
- Whether the 2D depiction and 3D graph agree after sanitization.
- New close contacts, distorted local geometry, and movement outside the edited region.
- Whether topology-dependent records—parameterization, docking mapping, or calculation results—were invalidated or remapped as expected.

A synchronous mutation error restores the transaction snapshot. If the chemistry is valid but optional coordinate polishing fails, the valid graph can remain committed and the cleanup error is recorded. Fused-aromatic edits that cannot preserve the aromatic graph are rejected rather than approximated.

Distance, angle, dihedral, atom translation, and graph-chemistry controls all have named actions. A representative graph edit follows; coordinate-only edits use `geometry.setInternalCoordinate` or `geometry.translateAtoms` and remain separate from `chemistry.finish`.

Agent skill contract: `edit_molecular_graph`

Contract element	Requirement
Goal	Apply an explicit, reviewable graph or local-geometry change while preserving atomic lineage and undoability.
Preconditions	The intended editable component is identified; no unrelated transaction is pending; the target atoms or bond are unambiguous.
Inputs	Selected stable atom identifiers; edit type and value; optional connected geometry path; intended transaction boundary.
Procedure	Inspect target; open or join a transaction; apply supported edits; finish once; inspect sanitized graph and local coordinates.

Continued on next page

Contract element	Requirement
State mutation	Graph edits remain provisional until Finish. Finish commits one new state and invalidates obsolete parameterization; Discard restores the starting state.
Evidence	Pre-edit identity, ordered public actions, changed atoms and bonds, sanitization outcome, cleanup outcome, and final inspection.
Success	The final graph encodes the intended chemistry, sanitizes successfully, and passes local visual and structural checks.
On failure	Keep the transaction pending for correction or discard it. Never bypass sanitization or inject coordinates directly.
Boundary	Local graph validity does not establish synthetic feasibility, tautomer preference, binding affinity, or global conformational plausibility.

```
await api.execute({action:"view.setMode", args:{mode:"build"}})
await api.execute({action:"build.setTool", args:{tool:"select"}})
await api.execute({action:"selection.replace", args:{atomIds:[anchorId]}})
await api.execute({action:"chemistry.addAtom",
  args:{attachedToAtomId:anchorId, element:"Cl"}})
await api.execute({action:"chemistry.finish", args:{}})
await api.execute({action:"geometry.setInternalCoordinate",
  args:{atomIds:[a,b,c,d], value:172.0, moveConnected:true}})
await api.execute({action:"session.inspect",
  args:{scope:"ligand", includeCoordinates:true, maximumAtoms:200}})
```

The serialized value `build` selects the user-facing Design workspace; it is retained for action-script compatibility.

B.5 Grow an analogue by adding and positioning fragments

Status. Current Design workspace; pose-aware extensions experimental.

The question. How do I add the substituent I have in mind while preserving the useful context of the parent structure?

Fragment growth combines a graph edit with an initial 3D placement. Those are separate scientific problems: the new molecule must first be chemically correct, and its coordinates must then be a plausible starting point for relaxation or pose maintenance. Molarium's Design workspace stages a fragment template and applies its attachment as one undoable edit. Subsequent atom, bond, charge, hydrogen, or stereochemical changes use the pending chemistry transaction described above.

Human workflow

1. If the parent is bound and its pose matters, capture the reference pose **before** beginning the topology change. Retain any explicit core or interaction choices needed for later pose maintenance.
2. Select the ligand attachment atom and stage the desired fragment or template. Confirm which atoms are new, which parent atom remains the anchor, and which bond is to be formed.
3. Apply the attachment as one undoable fragment edit. Perform any associated leaving-group, bond-order, formal-charge, hydrogen, or stereochemical changes as a subsequent pending chemistry transaction.
4. Use distance, angle, and dihedral controls to give the new group a sensible initial orientation. These controls establish a starting geometry; they do not score the binding site.
5. Finish any subsequent pending chemistry, then inspect the attachment, local valence, stereochemistry, ring geometry, clashes, and preserved parent coordinates. Relax the structure or invoke reference-guided pose maintenance as a separate step.

How should I judge the result?

A successful fragment operation means that the intended graph was built and sanitized. It does not mean that the chosen rotamer, ring conformation, or pocket orientation is preferred. For a solvent-exposed edit, local minimization may be enough to remove construction strain. For an edit that changes pocket contacts or releases a ring, use a workflow that samples alternatives and retains the relationship to the experimental reference.

Enumeration boundary. Development modules can represent bounded edit plans, but general combinatorial

enumeration, library management, automated campaign selection, and automatic MCS transfer of an independently imported analogue are not current workbench workflows.

Agent skill contract: `grow_fragment`

Contract element	Requirement
Goal	Construct one intended analogue and a reviewable starting geometry while retaining parent atom lineage and, when requested, reference-pose context.
Preconditions	Editable ligand and attachment atom identified; fragment chemistry specified; reference captured first if pose continuity matters.
Inputs	Parent and attachment atom identifiers; fragment/template; new bond; associated deletions, charge or stereochemical edits; optional geometry targets.
Procedure	Capture reference if needed; stage and attach the fragment; perform and finish any associated atom/bond transaction; set only the geometry needed for a starting pose; inspect; relax or sample separately.
State mutation	Attachment commits the fragment graph and an undo snapshot immediately. Finishing a later atom/bond transaction commits those additional changes. A later pose or optimization changes coordinates separately.
Evidence	Parent and product inspection, atom mapping, edit actions, transaction outcome, initial placement rationale, and any preserved reference/contact record.
Success	The attached analogue graph is correct on inspection; any subsequent chemistry transaction is sanitized; the initial geometry has no obvious construction pathology and is suitable for the chosen next workflow.
On failure	A failed attachment leaves the prior molecule in place. Correct or discard any subsequent pending chemistry transaction. Do not treat failed cleanup as permission to accept an invalid graph or arbitrary pose.
Boundary	Fragment growth is not combinatorial design, synthesis planning, global docking, or proof that the new group is energetically favored.

Generic fragment-template staging and attachment use `fragment.stage` and `fragment.attach`; individual atom-/bond edits and registered `designRoute.applyStep` operations remain available. For a registered, hash-pinned prospective route, an agent can stage a predefined product graph. `attachmentAtomId` is required only when that route declares designer-directed `spatialIntent`; the SOS1 route does not:

```
await api.execute({action:"fragment.stage",
  args:{smiles:"c1ccncc1", name:"pyridyl", attachmentIndex:0}})
await api.execute({action:"fragment.attach",
  args:{attachedToAtomId:anchorId}})

await api.execute({action:"designRoute.load",
  args:{routeId:"sos1-hit-only"}})
await api.execute({action:"view.setMode", args:{mode:"build"}})
await api.execute({action:"protein.prepare", args:{
  pH:7.4, histidine:"auto", repairMissingHeavy:true,
  ligandPolicy:"ccd", waterPolicy:"retain", gapPolicy:"cap"}})
await api.execute({action:"pose.captureReference",
  args:{mode:"propagate"}})
await api.execute({action:"designRoute.applyStep",
  args:{stepId:"scaffold-rewrite"}})
await api.execute({action:"designRoute.inspect", args:{}})
```

Registered routes constrain prospective inputs and permitted edits; they are not retrospective campaign ledgers.

B.6 Relax a structure, run bounded dynamics, and inspect motion

Status. Current and experimental methods with different scopes.

The question. I have made a structure. Does its geometry survive relaxation, and what moves during a short calculation?

Choose the method from the scientific question, not from a generic expectation that every engine answers the same thing. Molarium exposes several numerical pathways whose energies are not interchangeable. Record the selected model, solvent, constraints, and protocol before interpreting the result.

Question	Method	What it provides	Principal boundary
Small-molecule cleanup or short force-field motion	RDKit	Energy, geometry optimization, and short dynamics using the RDKit force-field path.	The reported 0.1 fs dynamics path is a workbench calculation, not a production trajectory.
Sage energy, relaxation, or bounded single-system MD	Direct WebGPU	Browser-GPU energy, forces, geometry, and saved frames from the numeric Sage system.	Experimental; f32; direct dynamics capped at 5,000 steps; no PBC or PME.
Many replicas of one topology	WebGPU ensemble	Homogeneous replica dynamics with energies, frames, timing, and constraint diagnostics.	Experimental; at most 512 atoms per replica, 1,024 replicas, and 100,000 steps; nonperiodic all-pairs model.
Electronic energy or geometry for an eligible neutral molecule	ANI-2x	Eight-model ANI-2x ensemble energy, spread, analytical forces, and optional optimized coordinates.	Experimental; connected neutral singlets with explicit H; H/C/N/O/S/F/Cl only; at most 96 atoms.
Reference calculation inside another workflow	OpenMM 8.2 Reference	Double-precision internal parameterization, scoring, relaxation, and validation path.	Internal subsystem, not a selectable general Simulate method.

Human workflow

1. Finish any pending chemistry. Confirm that the molecule is eligible for the chosen engine and, where required, can be completely parameterized.
2. In **Simulate**, choose the job and compatible method. Set the step count, vacuum or OBC2/ACE implicit solvent, X-H constraints where supported, saved frames, and ensemble options.
3. Run once. Inspect the engine or model identity, force field, charge model, solvent, constraints, initial and final energy, convergence or diagnostic status, elapsed time, and all warnings.
4. For dynamics, step through saved frames or replicas and inspect the region relevant to the design question. Display alignment does not overwrite raw calculation coordinates.
5. Review or export coordinates only when the result passes the declared acceptance check. Coordinate-producing jobs immediately update the current coordinates when they complete, and selecting or playing a saved frame writes that frame into the current molecule. Export the starting coordinates first when reversibility matters. Current-frame XYZ is available; saved frames are a bounded sample, and the workbench has no generic full-trajectory-file export.

Interpreting the model

Automatic Sage parameterization uses deterministic Gasteiger charges rather than AM1-BCC or NAGL. Unsupported elements, invalid chemistry, or missing required parameters fail instead of silently dropping terms. The exposed workflows are nonperiodic and do not provide PME, a barostat, explicit-solvent setup, generic production protein simulation, or binding free energy.

Agent skill contract: `relax_or_simulate`

Contract element	Requirement
Goal	Use a declared numerical model to test local geometry or bounded motion and retain enough metadata to interpret the result.
Preconditions	No chemistry transaction is pending; method eligibility and complete parameterization are established; an acceptance observable is specified.
Inputs	Job; method; steps; solvent; constraints; saved frames; fixed or movable atoms where supported; ensemble settings.

Continued on next page

Contract element	Requirement
Procedure	Validate eligibility; record model and protocol; export the starting coordinates when reversibility matters; run one compatible job; inspect energies, diagnostics, and frames.
State mutation	Energy jobs do not move the molecule. Successful coordinate-producing jobs immediately replace current coordinates, and selecting a saved frame also writes current coordinates; raw frames remain calculation records.
Evidence	Engine/model version and hashes where recorded, force field, charge model, solvent, constraints, timestep, energies, convergence/diagnostics, timing, frames, and warnings.
Success	The worker returns a finite, internally valid result and the predefined structural check is satisfied within the method's declared scope.
On failure	Keep the prior graph and coordinates; record the returned error; correct eligibility, options, or assets before retrying.
Boundary	A minimized pose or short stable trace is not an affinity estimate, equilibrium ensemble, free-energy calculation, or production trajectory.

The current public API exposes a bounded Design optimizer:

```
await api.execute({action:"view.setMode", args:{mode:"build"}})
await api.execute({action:"optimization.run",
  args:{method:"induced-fit-webgpu"}})
await api.execute({action:"session.inspect",
  args:{scope:"ligand", includeCoordinates:true, maximumAtoms:500}})
```

General Simulate jobs use `calculation.run`; the action accepts only the installed engine/job combinations and the same bounded options as the interface. Selecting a returned frame is explicit because it writes those coordinates into the current molecule:

```
await api.execute({action:"calculation.run", args:{
  job:"dynamics", method:"stormm",
  options:{stormmSystem:"current", steps:1000,
           savedFrameCount:20, replicaCount:32}}})
await api.execute({action:"calculation.selectFrame", args:{index:19}})
```

B.7 Explore conformations with Conformer Arena

Status. Experimental current surface.

The question. What other conformations can this molecule adopt, and how sensitive is the answer to the generation and refinement method?

Conformer search is an alternative-generation problem. Molarium separates seed generation, physical refinement, clustering, comparison, and selection so that a low-energy-looking structure is not confused with proof that the ensemble is complete or Boltzmann weighted.

Human workflow

1. Start from a chemically finished small molecule with explicit hydrogens and a sensible protonation state. Recenter the structure before tight STORMM positional comparisons, because the f32 force-evaluation coordinates lose precision far from the working origin.
2. Choose **WebGPU ensemble** and **Conformer search**. Set the requested conformer count, effort, clustering cutoff, solvent/constraint options, and whether to open the comparison arena.
3. Molarium generates deterministic RDKit conformer seeds, up to the implemented cap of 256. The STORMM protocol then performs staged relaxation, heating at 600 K, cooling through 450 K and 300 K, and final relaxation across homogeneous replicas.
4. Inspect the returned ensemble before selecting a representative: energy range, distinct RMSD clusters, cluster populations, recurring torsions, ring states, constraint diagnostics, and obvious duplicates or failures.
5. Use Conformer Arena to compare supported pathways on the same molecule and settings. Ask whether the methods discover the same basins, not merely whether they rank one shared conformer similarly.
6. Select a conformer only after inspection. Selection changes the displayed/current coordinates. Export the ensemble as SDF when it fits V2000 record limits; retain method and clustering metadata separately.

What does the result establish?

The workflow shows which basins the declared finite search found and how the supported methods behaved on that search. The anneal/minimize protocol is a search heuristic, so replica counts and cluster frequencies are not thermodynamic populations. Failure to find a conformation is not evidence that it is inaccessible, and energy differences from different force fields are not directly comparable.

Agent skill contract: `explore_conformers`

Contract element	Requirement
Goal	Generate, refine, cluster, and compare a finite set of conformational alternatives for one molecular topology.
Preconditions	Chemistry and protonation are finalized; molecule is eligible; intended use and comparison metric are stated.
Inputs	Conformer count; effort; cluster cutoff; seed; solvent and constraints; arena choice; acceptance or diversity criteria.
Procedure	Generate RDKit seeds; run declared refinement protocol; cluster and inspect; compare basins in Arena when requested; select or export without rewriting raw records.
State mutation	Search produces an ensemble and raw calculation evidence. Selecting a frame or conformer writes its coordinates into the current molecule.
Evidence	Seed and method identity, requested and returned counts, energies, cluster assignments, diagnostics, timings, and exported ensemble where applicable.
Success	A finite set of valid, distinct candidates is produced and its diversity is sufficient for the stated downstream purpose.
On failure	Report parameterization, eligibility, replica, overflow, or pair-work limits; reduce scope or escalate rather than hiding missing replicas.
Boundary	Not exhaustive conformational sampling, a Boltzmann ensemble, solution free energies, or proof of bioactive-conformation probability.

Conformer search and Arena use `calculation.run`; result selection uses `calculation.selectConformer`. Arena axes, filtering, and sorting are presentation controls exposed through `calculation.setConformerView`; they do not create new conformers:

```
await api.execute({action:"calculation.run", args:{
  job:"conformers", method:"stormm",
  options:{conformerCount:32, conformerEffort:"standard",
           conformerClusterRms:0.75, conformerArena:true}}})
await api.execute({action:"calculation.selectConformer", args:{rank:0}})
```

B.8 Predict a short single-chain protein structure

Status. Experimental current surface; connected mode only.

The question. Can Molarium create a protein structure from sequence, and is this the co-folding workflow?

This is not currently co-folding. The implemented workflow predicts one protein entity from one sequence. It does not accept a ligand for joint protein-ligand prediction, does not predict a multimer, and does not maintain a starting ligand pose. Calling it co-folding would overstate the present capability.

The Fold Protein panel combines a configured remote MSA service with local OpenFold ONNX inference. The sequence is transmitted to the selected MSA endpoint; model inference then runs in the browser, preferring WebGPU and falling back to WASM. Local Lab disables the external MSA path.

Human workflow

1. Enter one amino-acid sequence using the standard residue alphabet or X. The current maximum length is 128 residues, using 64- or 128-residue feature buckets.
2. Inspect and accept the configured MSA endpoint and network boundary. First use may require approximately 540 MB of model assets.
3. Run and monitor MSA and inference progress. The current browser model executes three recycle passes; this count is not user-configurable. Cancel aborts the active MSA/prediction controller.

4. On success, Molarium loads the prediction as the current molecule. Inspect chain completeness, residue mapping, gross geometry, and suitability for the downstream question before replacing other work.
5. Export the predicted structure as PDB if it must persist. Preparation and local relaxation are separate decisions.

The current path does not use templates or perform Amber relaxation. It creates no server-side job ledger. Fetch, archive, feature, model, or inference failure produces no partial prediction. A returned structure is a model output, not experimental evidence or a prepared simulation system.

Agent skill contract: `predict_protein_structure`

Contract element	Requirement
Goal	Predict coordinates for one short protein sequence using the declared MSA service and local OpenFold model.
Preconditions	Connected mode; configured MSA endpoint; sequence length at most 128; network and model-asset use accepted.
Inputs	One sequence and MSA endpoint; the current model uses three recycle passes.
Procedure	Validate sequence; disclose external transmission; obtain and validate MSA; run the matching ONNX bucket; inspect result; export before replacing it.
State mutation	A successful prediction replaces the current molecule. Failure does not load a partial structure.
Evidence	Input sequence and endpoint, model/bucket and executed recycle count, progress/failure record, prediction metadata, and exported PDB where required.
Success	A complete finite prediction is returned for the declared input and passes basic structural inspection for the intended next step.
On failure	Report MSA, archive, rate-limit, maintenance, timeout, feature, asset, or inference failure; do not fabricate a partial structure.
Boundary	No ligand co-folding, multimer prediction, templates, Amber relaxation, experimental validation, or production preparation.

The workflow is exposed as `protein.predict`; `protein.cancelPrediction` aborts the active request. The MSA endpoint remains an explicit network boundary rather than a hidden service call:

```
await api.execute({action:"protein.predict", args:{
  sequence:fastaText, msaEndpoint:"https://msa.example/api"}})
// while active:
await api.execute({action:"protein.cancelPrediction", args:{}})
```

B.9 Maintain an experimental pose while changing ligand chemistry

Status. Experimental current surface.

The question. I trust the experimental pose of my starting ligand. How can I search for a plausible pose of an edited analogue without discarding what I already know?

Reference-guided docking is a local analogue pose-maintenance workflow. It preserves stable atom identities and selected interactions from a prepared reference complex, searches edited groups and eligible edit-associated torsions, including explicitly released inherited atoms, rejects infeasible candidates, and ranks distinct feasible poses in a rigid local receptor site. It is intentionally narrower than global docking.

Human workflow

1. Prepare and inspect the reference protein-ligand complex. In **Design**, capture the reference before editing. The capture stores ligand lineage, receptor atoms within 8 Å, reference contacts, selected hydrogen bonds, settings, and provenance.
2. Use the default **Pose propagation** mode to map surviving reference atoms and seed the edited region. Use **Expert selected core** only when you can justify at least three connected, non-collinear heavy atoms as a fixed reference.
3. Edit and finish the analogue. Review remapped or unavailable contacts, transformed ring regions, atom mapping, and any constraints before starting the search.

4. Run the constrained search. Molarium verifies receptor and contact integrity, releases eligible transformed rings, generates deterministic feature and torsion seeds, freshly parameterizes the edited ligand, relaxes feasible candidates, and scores the distinct results.
5. Inspect the ranked candidates, not only rank 1. Compare mapped-core displacement, preserved or replaced interactions, ligand strain, clashes, ring and side-chain choices, and the reasons candidates were rejected.
6. Apply one candidate explicitly. Candidate generation does not change the authoritative molecule. Application rechecks topology and receptor integrity before replacing coordinates. Export the labbook as JSON and human-readable Markdown/notes.

What is being scored?

The ranker uses a rigid receptor site within 8 Å, Lennard-Jones and Coulomb ligand-receptor cross terms with relative dielectric 4, and relative vacuum ligand strain. It omits general receptor relaxation, water displacement, entropy, binding free energy, and broad macrocycle or fused-ring concerted search. The separate induced-fit Design optimizer is not part of this ranker and is not an affinity calculation. Required hydrogen bonds are acceptance conditions for constrained ligand search, not restraints in ligand cleanup, Pocket relax, or Induced-fit pocket. The latter two also differ in whether nearby backbone atoms may move and which ligand atoms remain protected. Adjacent information buttons explain these policies and the available choices. The UI uses v5 feature seeding and automatic worker execution; legacy seeding and execution overrides remain API-only. Weak covalent-fluorine contact hypotheses are optional by default.

Negative results are results. An invalid core, altered receptor, unmappable required atom, unavailable contact feature, parameterization failure, or absence of feasible candidates produces an explicit failed or negative record. Molarium does not silently apply the least-bad pose.

Agent skill contract: `maintain_reference_pose`

Contract element	Requirement
Goal	Generate and review plausible local poses for one edited analogue while retaining explicit relationships to a trusted reference complex.
Preconditions	Prepared complex; trusted reference pose; reference captured before edit; chemistry finished; receptor unchanged; required contacts available.
Inputs	Pose-propagation or expert-core mode; selected core or contacts; cleanup/contact policy; candidate count; analogue graph.
Procedure	Capture; edit; finish; verify mapping and receptor integrity; generate and score feasible candidates; inspect alternatives; apply one explicitly or retain a negative result.
State mutation	Capture and search create reference and candidate records. Only Apply replaces the current coordinates.
Evidence	Protocol/settings snapshot, hashes, atom mapping, contact decisions, constraints, rejected candidates, scores, selected pose, and chained labbook.
Success	At least one feasible distinct candidate survives the declared constraints and a human or agent can justify any applied selection from the recorded evidence.
On failure	Apply nothing; preserve the negative labbook; repair the reference, mapping, constraints, or chemistry before rerunning.
Boundary	Local analogue pose maintenance only—not global docking, automatic MCS pose transfer, receptor-ensemble docking, or affinity/free-energy prediction.

```
await api.execute({action:"pose.captureReference",
  args:{mode:"propagate"}})
// perform and finish the intended molecular-graph edit
await api.execute({action:"pose.refine",
  args:{searchChains:16}})
await api.execute({action:"pose.apply", args:{index:0}})
await api.execute({action:"session.inspect",
  args:{scope:"pocket", includeCoordinates:true, maximumAtoms:500}})
```

For a receptor-side-chain alternative, the required order is:

```
await api.execute({action:"pose.enumerateSidechainRotamers",
```

```
  args:{receptorAtomId:sidechainAtomId, maximumCandidates:32}})
await api.execute({action:"pose.applySidechainRotamer", args:{index:5}})
await api.execute({action:"pose.updateReceptorReference", args:{}})
await api.execute({action:"pose.refine", args:{searchChains:64}})
```

Enumeration itself remains non-mutating; the selected receptor coordinates are applied explicitly and then made the new reference before another ligand-pose search.

B.10 Record, replay, and hand off a design process

Status. Current action/replay and Design History surfaces.

The question. I have done something scientifically useful. What must I export so that another scientist or an agent can understand and reproduce it?

Molarium records several different kinds of evidence. They answer different questions and should not be treated as interchangeable. The live action audit says what was requested; a calculation or workflow record says what method ran and what it returned; an action script says how to replay completed public operations; and Design History represents a durable sequence of content-addressed states and decisions. The current workbench does not automatically combine every scientific record into one complete project archive.

Record	Question answered	Persistence and limit
Chemist Actions audit	What recognized public action was requested, with what arguments and outcome?	Session memory; serialized calls; cleared with the scene unless exported through a script path.
Preparation or calculation record	What system, model, settings, energies, frames, warnings, or audit produced this result?	Attached to the in-memory state and available through workflow-specific reports or exports.
Docking labbook	How were mapping, feasibility, scoring, rejection, and selection handled?	Exportable JSON and Markdown/notes with chained hashes.
Designer action script	Which completed public actions should be replayed?	Exportable JSON; replay invokes the same guarded routes; not a complete UI or binary session snapshot.
Registered design route	What prospective, hash-pinned edit/evaluation sequence is permitted?	Validated route schema; distinct from retrospective history.
Design History campaign	What content-addressed structures, decisions, commits, and branches form the design history?	Live browser surface with locally persisted commits, branches, merges, decisions, integrity events, import, and export.

Replay workflow

Designer Moves import begins from a blank canvas and performs script-level validation of schema, recognized action names, binding order, and forbidden shortcut keys. Individual action arguments and molecular preconditions are checked when each step executes. Conversion from an audit includes only completed public actions and excludes campaign administration. A standalone script therefore begins with an explicit reproducible loader such as `session.loadStructure`, `session.loadIdentifier`, `session.loadFixture`, or `designRoute.load`; it cannot depend on an unrecorded starting scene.

1. Export any unsaved molecular or workflow records before importing a script; import intentionally clears the existing scene.
2. Create a Designer action script from completed recognized public actions. Failed actions remain audit evidence but are not silently converted into successful replay steps.
3. Import the validated script, restart it, and Play or step through it. Replay uses the same Chemist Actions routes, runtime validations, and workers as manual operation and produces a separate replay audit.
4. Compare the resulting graph, coordinates, decisions, and errors with the intended endpoint. Export the replay audit and scientific records needed by the recipient.
5. For a registered story or route, verify schema, referenced content, and SHA-256 before execution. Treat a hash chain as integrity evidence, not an authenticated author identity or proof of scientific truth.

Replay provides play/pause, previous/next-step review, stable captions, and a separate audit. Failed audit entries remain evidence but are not converted silently into successful replay steps.

Live Design History

Design History is a live browser feature. `campaign.create` creates a locally persisted campaign and atomically commits the visible molecule. `campaign.commitCurrent` stores a complete graph-and-coordinate snapshot together with completed public actions since the prior commit. Branch switching restores the exact head molecule and refuses to proceed when the visible molecular state is dirty. Merge records the currently visible result with two parents; it is not an automatic structural merge. Decisions attach disposition, rationale, reason codes, and optional evidence to a commit. Verification recomputes content hashes, the append-only event chain, commit relationships, and branch heads derived from that chain.

Campaign JSON and the active branch are stored in browser IndexedDB. Reload makes the saved campaign available to resume without replacing the visible molecule. Explicit Resume restores the graph and coordinates at the active branch head; it does not restore parameter tables, docking candidates, calculation frames, view state, or the complete workbench session. `campaign.import` verifies the canonical JSON and requires a restorable graph/coordinate snapshot before mutating live state; `campaign.export` returns that canonical JSON. Closing a campaign clears the active pointer without deleting its commits. A live commit can attach a script of completed molecular Chemist Actions; the complete molecular snapshot remains authoritative. Campaign-management actions are excluded from molecule replay scripts so replay cannot recursively administer its own history.

Choose the export that matches the handoff

Export	Carries	Does not carry
XYZ current frame	Displayed coordinates	Full topology metadata, history, or provenance
SDF conformer ensemble	Exportable conformer graph and coordinates	Unrelated session state or complete search history
PDB protein	Imported or predicted protein coordinates and supported annotations	Complete application or preparation provenance
Preparation JSON	Policies, issues, audit, and readiness status	A production-readiness guarantee
Docking JSON/-Markdown	Settings, mapping, candidates, failures, hashes, and selection	Author authentication or binding free energy
Action script JSON	Public actions and bindings	Arbitrary code, direct coordinates, or complete view state
PNG or share URL	A visual snapshot or current/registered URL	A reusable molecule or serialized live session

Agent skill contract: `export_and_handoff`

Contract element	Requirement
Goal	Preserve the smallest complete set of molecular, procedural, numerical, and decision records needed by the next scientist or agent.
Preconditions	The authoritative result and its intended downstream use are identified; unsaved state has not yet been cleared or replaced.
Inputs	Recipient and purpose; required molecular format; action/replay needs; calculation, preparation, docking, or campaign evidence to retain.
Procedure	Inventory records; export reusable molecular state and method evidence; create an action script when replay is required; commit decision history when project continuity is required; verify hashes and replay; describe missing pieces explicitly.
State mutation	Export is non-mutating. Script import is destructive to the unsaved scene and must follow export of work that must be retained.
Evidence	Exported files, schema and version identifiers, hashes where recorded, replay audit, and a statement of assumptions and unresolved failures.
Success	The recipient can identify the starting state, reproduce guarded actions where supported, inspect the method and result, and distinguish accepted decisions from candidates.
On failure	Stop on schema, action, binding, hash, or referenced-content mismatch. Do not describe a partial replay as equivalent to a completed audit.
Boundary	No complete project bundle or automatic capture of browser-native transport, calculation binaries, or every transient view gesture. An action script is not a substitute for the authoritative snapshot.

```
await api.execute({action:"campaign.create", args:{
  campaignId:"sos1-local-design", title:"SOS1 local design",
  initialCommitMessage:"Register 5OVE/AXE starting state"}})
// chemist or agent performs guarded molecular actions
await api.execute({action:"campaign.commitCurrent", args:{
  message:"Accept Phe890-out design state", tags:["prospective"]}})
await api.execute({action:"campaign.recordDecision", args:{
  disposition:"progressed",
  rationale:"No new clashes after pocket-state search"}})
await api.execute({action:"campaign.verify", args:{}})
```

B.11 How skill contracts, API actions, and Design History fit together

Status. Current conceptual and executable relationship.

The question. How does a written agent contract become an authorized action and then a durable scientific decision?

The three layers serve different purposes and should not be collapsed:

1. **The skill contract defines scientific intent.** It states preconditions, permissible operations, required evidence, success, failure, and escalation. Here these names are proposed narrative operating contracts stored with the manuscript. They are not installed runtime skills, Chemist Actions, schemas, or executable enforcement.
2. **The Chemist Actions API defines executable authority.** `api.describe()` lists installed action names and bounded arguments. Calls are validated, serialized through one queue, and audited with sequence, status, result or error, and duration. An agent cannot gain extra molecular authority by phrasing a request differently.
3. **Design History defines durable meaning over time.** It commits complete molecular states and can link them to an explicitly partial public-action script, branches, decisions, evidence, actors, and append-only integrity records. The snapshot answers "what state was accepted?"; the action script answers "which guarded operations were captured and can be replayed?"; and the decision answers "why was it accepted?"

The practical agent pattern is:

```
const manifest = api.describe();          // discover installed authority
await api.execute({action:"session.inspect", args:{scope:"all"}})
await api.execute({action:"geometry.setInternalCoordinate", args:{
  atomIds:[a,b,c,d], value:172.0, moveConnected:true}})
await api.execute({action:"session.inspect", args:{scope:"all"}})
await api.execute({action:"campaign.commitCurrent",
  args:{message:"Accepted state after declared check"}})
await api.execute({action:"campaign.recordDecision",
  args:{disposition:"progressed", rationale:"..."}})
await api.execute({action:"campaign.verify", args:{}})
```

The manifest is the executable authority: a narrative skill contract cannot add an action, relax argument validation, or bypass a molecular precondition. Scientific review remains necessary even when an operation is exposed to both interface and agent.

Handoff rule. Export the structure when the recipient needs coordinates; export the calculation record when the recipient must judge the method; export the Designer Moves script together with its required starting state when the recipient must replay operations; and commit to Design History when the recipient must recover branches, decisions, and accepted molecular states. Hashes detect alteration of the serialized record. They do not establish authorship, chemical equivalence, or scientific validity.

The appendix in one sentence. For every task: make the molecular state explicit, choose a method whose scope matches the question, inspect before applying a candidate, retain the evidence needed to reproduce the decision, and stop when the declared scientific boundary is crossed.

Capabilities deliberately outside these workflows

The current workbench does not provide missing-loop construction, membrane or general metal preparation, covalent-ligand parameterization, periodic explicit solvent, PME, production-scale trajectories, binding free energy, global docking, general remote CUDA execution, GFN/xTB, general combinatorial enumeration, or automatic campaign

selection. Their absence is part of the operating contract rather than a prompt to approximate them silently.

This task-centered appendix intentionally omits maintainer file paths, source-line numbers, worker lifecycle details, JSON nesting limits, deployment internals, and the complete repository test list. Those remain valuable implementation and audit documentation, but they are not required for a scientist or agent to understand how to perform and judge the workflows above.

References

[1] Bran, A. M.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; Schwaller, P. Augmenting Large Language Models with Chemistry Tools. *Nature Machine Intelligence* **2024**, *6*, 525–535.

[2] Boiko, D. A.; MacKnight, R.; Kline, B.; Gomes, G. Autonomous Chemical Research with Large Language Models. *Nature* **2023**, *624*, 570–578.

[3] Pham, T. D.; Tanikanti, A.; Keçeli, M. ChemGraph as an Agentic Framework for Computational Chemistry Workflows. *Communications Chemistry* **2026**, *9*, 33.

[4] MacKnight, R.; Boiko, D. A.; Regio, J. E.; Gallegos, L. C.; Neukomm, T. A.; et al. Rethinking Chemical Research in the Age of Large Language Models. *Nature Computational Science* **2025**, *5*, 715–726.

[5] Haas, A.; Rossberg, A.; Schuff, D. L.; et al. Bringing the Web up to Speed with WebAssembly. *Proc. PLDI* **2017**, 185–200.

[6] W3C GPU for the Web Working Group. *WebGPU Specification*, 2025.

[7] Riniker, S.; Landrum, G. A. Better Informed Distance Geometry: Using What We Know to Improve Conformation Generation. *J. Chem. Inf. Model.* **2015**, *55*, 2562–2574.

[8] Boothroyd, S.; Madin, O. C.; Mobley, D. L.; et al. Development and Benchmarking of Open Force Field 2.0.0: The Sage Small-Molecule Force Field. *J. Chem. Theory Comput.* **2023**, *19*, 3251–3275.

[9] Eastman, P.; Galvelis, R.; Peláez, R. P.; et al. OpenMM 8: Molecular Dynamics Simulation with Machine Learning Potentials. *J. Phys. Chem. B* **2024**, *128*, 109–116.

[10] Cerutti, D. S.; Wiewiora, R. P.; Boothroyd, S.; Sherman, W. STORMM: Structure and Topology Replica Molecular Mechanics for Chemical Simulations. *J. Chem. Phys.* **2024**, *161*, 032501.

[11] Devereux, C.; Smith, J. S.; Huddleston, K. K.; et al. Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *J. Chem. Theory Comput.* **2020**, *16*, 4192–4202.

[12] Ahdritz, G.; Bouatta, N.; Floristean, C.; et al. OpenFold: Retraining AlphaFold2 Yields New Insights into Its Learning Mechanisms and Capacity for Generalization. *Nature Methods* **2024**, *21*, 1514–1524.

[13] Ryckaert, J.-P.; Ciccotti, G.; Berendsen, H. J. C. Numerical Integration of the Cartesian Equations of Motion of a System with Constraints: Molecular Dynamics of n-Alkanes. *J. Comput. Phys.* **1977**, *23*, 327–341.

[14] Andersen, H. C. RATTLE: A "Velocity" Version of the SHAKE Algorithm for Molecular Dynamics Calculations. *J. Comput. Phys.* **1983**, *52*, 24–34.

[15] Cohen-Boulakia, S.; Belhajjame, K.; Collin, O.; et al. Scientific Workflows for Computational Reproducibility in the Life Sciences: Status, Challenges and Opportunities. *Future Gener. Comput. Syst.* **2017**, *75*, 284–298.

[16] Pritchard, N. J.; Wicenec, A. Formal Definition and Implementation of Reproducibility Tenets for Computational Workflows. *Future Gener. Comput. Syst.* **2025**, *166*, 107684.

[17] Wilkinson, M. D.; Dumontier, M.; Aalbersberg, I. J.; et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Sci. Data* **2016**, *3*, 160018.

[18] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 1. *Drug Discov. Today* **2021**, *26*, 511–524.

[19] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 2. *Drug Discov. Today* **2021**, *26*, 1040–1052.

[20] Wigh, D. S.; Goodman, J. M.; Lapkin, A. A. A Review of Molecular Representation in the Age of Machine Learning. *WIREs Comput. Mol. Sci.* **2022**, *12*, e1603.

[21] Bainbridge, L. Ironies of Automation. *Automatica* **1983**, *19*, 775–779.

[22] W3C GPU for the Web Working Group. *WebGPU Shading Language*, 2026. https://www.w3.org/TR/WGSL/.

[23] Wang, S.; Witek, J.; Landrum, G. A.; Riniker, S. Improving Conformer Generation for Small Rings and Macrocycles Based on Distance Geometry and Experimental Torsional-Angle Preferences. *J. Chem. Inf. Model.* **2020**, *60*, 2044–2058.

[24] Halgren, T. A. Merck Molecular Force Field. I. Basis, Form, Scope, Parameterization, and Performance of MMFF94. *J. Comput. Chem.* **1996**, *17*, 490–519.

[25] Rappé, A. K.; Casewit, C. J.; Colwell, K. S.; Goddard, W. A.; Skiff, W. M. UFF, a Full Periodic Table Force Field for Molecular Mechanics and Molecular Dynamics Simulations. *J. Am. Chem. Soc.* **1992**, *114*, 10024–10035.

[26] Gasteiger, J.; Marsili, M. Iterative Partial Equalization of Orbital Electronegativity—A Rapid Access to Atomic Charges. *Tetrahedron* **1980**, *36*, 3219–3228.

[27] Eastman, P.; Swails, J.; Chodera, J. D.; et al. OpenMM 7: Rapid Development of High Performance Algorithms for Molecular Dynamics. *PLoS Comput. Biol.* **2017**, *13*, e1005659.

[28] Onufriev, A.; Bashford, D.; Case, D. A. Exploring Protein Native States and Large-Scale Conformational Changes with a Modified Generalized Born Model. *Proteins* **2004**, *55*, 383–394.

[29] Glaser, J.; Nguyen, T. D.; Anderson, J. A.; et al. Strong Scaling of General-Purpose Molecular Dynamics Simulations on GPUs. *Comput. Phys. Commun.* **2015**, *192*, 97–107.

[30] Hillig, R. C.; Sautier, B.; Schroeder, J.; et al. Discovery of Potent SOS1 Inhibitors That Block RAS Activation via Disruption of the RAS–SOS1 Interaction. *Proc. Natl. Acad. Sci. U.S.A.* **2019**, *116*, 2551–2560.